

Faculty of Engineering

Department of Informatics Engineering

Software Engineering and Information Systems



Integrated Social Media Platform (NOONIFY)

A junior project report - submitted to complete the requirements for obtaining a bachelor's degree in informatics engineering – Software Engineering and Information Systems

Prepared by

Sara Eyad Attah

Aya Al-salamat

Supervised by

Dr. Eng. Akram Massouh

January 2026

منصة تواصل اجتماعي متكامل

تقرير مشروع فصلي قَدَم استكمالاً لمتطلبات الحصول على درجة
البكالوريوس في هندسة المعلوماتية - هندسة البرمجيات ونظم المعلومات

إعداد

ساره اياد عطا

آيه هيثم السلامات

اشراف

الدكتور. أكرم مَسُوح

SUPERVISION CERTIFICATION

I Certify that the preparation of this project entitled

[**Integrated Social Media Platform (NOONIFY)**]

Prepared by

[Sara Eyad Atta - Aya Alsalamat]

was made under my supervision at Faculty of Informatics Engineering in partial Fulfillment of the
Requirements for the Degree of Bachelor of Software Engineering and Information System

Name:

Signature:

Date:

الملخص

يهدف هذا المشروع إلى تصميم منصة تواصل اجتماعي متكاملة تتيح للمستخدمين إنشاء حساباتهم، مشاركة المحتوى النصي والمرئي، والتفاعل مع الآخرين بطريقة آمنة وسلسة، يعتمد النظام على MERN Stack (MongoDB, Express.js, React.js, Node.js) لتغطية الجوانب الأمامية والخلفية، إضافة إلى أدوات داعمة مثل Clerk لإدارة الحسابات وتوثيق المستخدمين، وImageKit لمعالجة وتحسين الصور، وInngest لإدارة المهام الخلفية المجدولة.

توفّر المنصة مجموعة واسعة من الوظائف تشمل التسجيل والدخول (Sign Up / Sign In)، إدارة الملف الشخصي (Profile Management)، إنشاء المنشورات (Posts) والقصص (Stories)، التفاعل الاجتماعي عبر الإعجابات والتعليقات والمتابعة (Follow / Unfollow)، بالإضافة إلى نظام مراسلة فورية (Real-time Chat).

كما تتيح المنصة البحث والاكتشاف (Search & Discover)، الإشعارات اللحظية (Real-time Notifications)، وإدارة الوسائط باستخدام حلول تخزين سحابية فعّالة.

من الناحية التقنية، يتميز النظام بالمرونة العالية (Scalability) بفضل بنية Node.js وMongoDB، مع واجهة مستخدم تفاعلية وسهلة الاستخدام مبنية بـ React.js. كما تم الاهتمام بعوامل الأمان (Security) باستخدام HTTPS، وضمان الاعتمادية (Reliability) وقابلية الصيانة (Maintainability) من خلال تصميم مكونات واضحة ومنفصلة.

بشكل عام، يقدم المشروع نموذجًا عمليًا متكاملًا لتطبيق تواصل اجتماعي حديث، يربط بين تجربة المستخدم الممتعة والأداء القوي المدعوم بأحدث تقنيات Full Stack Development.

Abstract

This project aims to develop a comprehensive social media platform that enables users to create accounts, share text and media content, and interact with others in a secure and seamless environment. The system is built using the MERN Stack (MongoDB, Express.js, React.js, Node.js), along with supporting tools such as Clerk for authentication and account management, ImageKit for media optimization, and Inngest for background task scheduling.

The platform provides a wide range of functionalities including Sign Up / Sign In, Profile Management, creating and managing Posts and Stories, engaging with others through Likes, Comments, and Follow/Unfollow, and real-time messaging (Real-time Chat). It also includes Search & Discover, Real-time Notifications, and efficient Media Management using cloud storage services.

Technically, the system ensures Scalability with Node.js and MongoDB, offers a highly interactive user interface built with React.js, and maintains strong Security standards via HTTPS. It emphasizes Reliability and Maintainability through modular code structure and clear component separation.

Overall, the project presents a fully functional and modern social networking application that combines an enjoyable user experience with robust technical performance, showcasing practical Full Stack Development skills

Table of Contents

4	المُلخَص
5	Abstract
1	1. الفصل الأول: المقدمة
2	1.1 أهمية المشروع
3	1.2 الهدف من المشروع
4	1.3 الدراسات المرجعية لمنصة التواصل الاجتماعي
4	1.3.1 الدراسة الأولى
5	1.3.2 الدراسة الثانية
6	1.3.3 جدول مقارنة بين الدراسات المرجعية
7	1.4 الجدول الزمني Gantt Chart
7	1.5 المنهجية المتبعة في تنفيذ المشروع (منهجية SCRUM)
8	1.5.1 تطبيق منهجية SCRUM في مشروع منصة التواصل الاجتماعي
9	1.6 المعمارية (System Architecture)
9	1.6.1 أولاً: معمارية Client-Server
9	1.6.1.1 مزايا معمارية Client-Server
11	1.6.2 ثانياً: معمارية MVC (Model – View – Controller)
13	1.6.2.1 إيجابيات معمارية MVC
15	1.6.3 لربط بين Client-Server و MVC في المشروع
17	1.7 الأدوات المستخدمة
17	1.7.1 أدوات الواجهة الخلفية
18	1.7.2 أدوات الواجهة الامامية
18	1.7.3 أدوات قاعدة البيانات
19	1.7.4 ملخص الأدوات المستخدمة
20	2. الفصل الثاني : الدراسة التحليلية
21	2.1 المتطلبات العامة (General Requirements)
21	2.2.1 نظام التسجيل والدخول (Authentication System)
221	2.2.2 إدارة الملف الشخصي (User Profile Management)

22	إدارة المنشورات (Posts Management)
22	إدارة القصص (Stories / Status System)
22	نظام المراسلة (Real-time Chat & Messages)
23	نظام العلاقات الاجتماعية (Connections System)
23	نظام البحث والاكتشاف (Search & Discover System)
23	نظام الإشعارات (Notifications System)
24	إدارة الوسائط (Media Management)
24	النظام الخلفي (Backend Operations)
24	المتطلبات غير الوظيفية (Non-Functional Requirements)
25	المتطلبات التي سوف بالفصلي
27	مخطط ال Use Case Diagram
28	توصيف حالات الاستخدام Use Case Specifications
28	مخططات النشاط Activity Diagrams
28	مخططات التسلسل Sequence Diagrams
64	الفصل الثالث : الدراسة التصميمية
65	3.1 مخطط sitemap
66	3.2 مخطط قاعدة البيانات ERD
67	3.3 مخطط الصفوف Class Diagram
66	3.4 جدول Test Cases
69	3.5 جدول RTM (Requirements Traceability Matrix)
71	4. الفصل الرابع : تنفيذ المشروع
72	4.1 الواجبات
79	4.2 الخاتمة
80	4.3 الآفاق المستقبلية (Future Work)

List of Tables

6	جدول 1.1 مقارنة بين الدراسات المرجعية
28	جدول 2.2 توصيف حالة تسجيل الحساب
30	جدول 2.3 توصيف حالة تسجيل الدخول
33	جدول 2.4 توصيف حالة الدخول الآمن
34	جدول 2.5 توصيف حالة تسجيل الخروج
36	جدول 2.6 توصيف حالة عرض الواجهة الأساسية
37	جدول 2.7 توصيف حالة التعديل على الحساب
40	جدول 2.8 توصيف حالة عرض معلومات الحسابات
42	جدول 2.9 توصيف حالة البحث عن الأصدقاء
43	جدول 2.10 توصيف حالة نشر منشور جديد
45	جدول 2.11 توصيف حالة عرض المنشورات
48	جدول 2.12 توصيف حالة الإعجاب بالمنشورات
50	جدول 2.13 توصيف حالة التعليق على المنشورات
52	جدول 2.14 توصيف حالة مشاركة المنشورات
54	جدول 2.15 توصيف حالة إضافة صديق
56	جدول 2.16 توصيف معرفة حالة طلب الصداقة
58	جدول 2.17 توصيف حالة إضافة قصة جديدة
60	جدول 2.18 توصيف عرض القصص المتاحة للمتابعين
62	جدول 2.19 توصيف عرض قائمة المشاهدين
68	جدول 3.20 Test Cases ال
69	جدول 3.21 ال RTM

List of Figuers

29	2.2 مخطط النشاط حالة تسجيل الحساب
30	2.3 مخطط التسلسل حالة تسجيل الحساب
32	2.4 مخطط النشاط حالة تسجيل الدخول
32	2.5 مخطط التسلسل حالة تسجيل الدخول
35	2.6 مخطط التسلسل حالة تسجيل الخروج
35	2.7 مخطط النشاط حالة تسجيل الخروج
37	2.8 مخطط التسلسل حالة عرض الواجهة الأساسية
37	2.9 مخطط النشاط حالة عرض الواجهة الأساسية
39	2.10 مخطط التسلسل حالة التعديل على الحساب
39	2.11 مخطط النشاط حالة التعديل على الحساب
41	2.12 مخطط النشاط حالة عرض معلومات الحسابات
41	2.13 مخطط النشاط حالة عرض معلومات الحسابات
43	2.14 مخطط النشاط حالة البحث عن الأصدقاء
43	2.15 مخطط التسلسل حالة البحث عن الأصدقاء
45	2.16 مخطط التسلسل حالة نشر منشور جديد
45	2.17 مخطط النشاط حالة نشر منشور جديد
47	2.18 مخطط النشاط حالة عرض المنشورات
47	2.19 مخطط التسلسل حالة عرض المنشورات
49	2.20 مخطط التسلسل حالة الإعجاب بالمنشورات
49	2.21 مخطط النشاط حالة الإعجاب بالمنشورات
51	2.22 مخطط النشاط حالة التعليق على المنشورات
51	2.23 مخطط التسلسل حالة التعليق على المنشورات
53	2.24 مخطط النشاط حالة مشاركة المنشورات
53	2.25 مخطط التسلسل حالة مشاركة المنشورات
55	2.26 مخطط التسلسل حالة إضافة صديق
55	2.27 مخطط النشاط حالة إضافة صديق
57	2.28 مخطط التسلسل معرفة حالة طلب الصداقة
57	2.29 مخطط النشاط معرفة حالة طلب الصداقة
59	2.30 مخطط التسلسل حالة إضافة قصة جديدة
59	2.31 مخطط النشاط حالة إضافة قصة جديدة
61	2.32 مخطط النشاط حالة عرض القصص المتاحة للمتابعين
61	2.33 مخطط التسلسل حالة عرض القصص المتاحة للمتابعين
63	2.34 مخطط النشاط عرض قائمة المشاهدين
63	2.35 مخطط التسلسل عرض قائمة المشاهدين

List of Forms

8		Scrum Methodology	الشكل ال 1.1
11		Client_Server	الشكل ال 1.2
13		MVC	الشكل ال 1.3
14		دمج بين المماريتين	الشكل 1.4
27		usecase diagram	الشكل 2.5
65		Sit Map	الشكل 3.6
66		ERD	الشكل ال 3.7
67		Class Diagram	الشكل ال 3.8
72		واجهة تسجيل حساب جديد	الشكل (1.1) 4.9
73		واجهة تسجيل الدخول	الشكل (1.2) 4.10
74		زر تسجيل الخروج	الشكل (1.3) 4.11
74		عرض المعلومات الأساسية للمستخدم	الشكل (1.4) 4.12
76		رفع قصة جديدة	الشكل (1.5) 4.13
75		إنشاء منشورات جديد	الشكل (1.6) 4.14
77		عرض ملفات المستخدمين الآخرين	الشكل (1.7) 4.15
78		تعديل الملف الشخصي	الشكل (1.8) 4.16
78		عرض القصص	الشكل (1.9) 4.17
78		عرض منشورات الآخرين	الشكل (1.10) 4.18

1. الفصل الأول: المقدمة

1.1 أهمية المشروع

تتبع أهمية هذا المشروع من الحاجة المتزايدة إلى منصات تواصل اجتماعي حديثة وآمنة تتيح للمستخدمين التفاعل ومشاركة المحتوى بسهولة وكفاءة، ومع التطور السريع في تقنيات الويب وازدياد الاعتماد على التطبيقات الرقمية في التواصل اليومي، أصبح من الضروري تطوير نظام تواصل اجتماعي متكامل يجمع بين الأداء العالي وتجربة المستخدم المميزة.

يساهم المشروع في تعزيز مهارات المطورين في مجال Full Stack Development من خلال تطبيق عملي يعتمد على تقنيات (MongoDB, Express.js, React.js, Node.js) MERN Stack، والتي تمكن من بناء نظام متكامل يغطي جميع الجوانب: Front-End لتصميم واجهة تفاعلية، و Back-End لمعالجة الطلبات وإدارة البيانات، و Database لتخزين المعلومات بطريقة فعالة وآمنة.

تكمُن أهمية المشروع أيضًا في كونه يدمج مفاهيم وتقنيات حديثة مثل Real-time Communication، وإدارة الوسائط باستخدام ImageKit، ونظام توثيق المستخدمين الآمن بواسطة Clerk، بالإضافة إلى إدارة المهام الخلفية باستخدام Inngest.

وبذلك، لا يقتصر المشروع على كونه تجربة تعليمية فحسب، بل يشكّل نموذجًا متكاملًا يمكن تطويره مستقبلاً ليصبح منصة تواصل اجتماعي حقيقية تتميز بالأمان، والاعتمادية، وسهولة الاستخدام، وتواكب التطورات التقنية الحديثة في عالم الويب.

1.2 الهدف من المشروع

يهدف هذا المشروع إلى تطوير منصة تواصل اجتماعي متكاملة تتيح للمستخدمين إنشاء حساباتهم، إدارة ملفاتهم الشخصية، ومشاركة المحتوى النصي والمرئي بطريقة آمنة وسلسة. كما يسعى إلى توفير بيئة تفاعلية تجمع بين التصميم الجذاب، الأداء العالي، وسهولة الاستخدام عبر مختلف الأجهزة والمنصات.

يتمثل الهدف الأساسي في بناء تطبيق ويب يعتمد على MERN Stack (MongoDB, Express.js, React.js, Node.js) لتغطية جميع طبقات النظام من الواجهة الأمامية (Front-End) وحتى النظام الخلفي (Back-End) وقاعدة البيانات (Database). كما يهدف المشروع إلى تعزيز الجوانب التقنية من خلال دمج أدوات وخدمات حديثة مثل Clerk لإدارة المستخدمين وتوثيق الحسابات، وImageKit لمعالجة الصور وتحسين أدائها، وInngest لتنفيذ المهام الخلفية المجدولة مثل الإشعارات أو حذف البيانات المؤقتة. ومن الأهداف المهمة كذلك، تطبيق مفاهيم Real-time Features مثل المراسلة الفورية (Real-time Chat) والإشعارات اللحظية (Real-time Notifications) لتوفير تجربة استخدام ديناميكية وتفاعلية. إضافة إلى ذلك، يسعى المشروع إلى بناء واجهة استخدام متجاوبة (Responsive UI) باستخدام React.js و CSS لتتيح سهولة التنقل وتفاعل المستخدمين بسلاسة.

باختصار، يهدف المشروع إلى تقديم نموذج تطبيقي متكامل يربط بين الجانب الأكاديمي والعمل، ويُمكن المطورين من تطبيق مهاراتهم في بناء Social Media Platform احترافية تُبرز المفاهيم الأساسية لتقنيات الويب الحديثة وأفضل ممارسات التطوير المتكامل.

1.3 الدراسات المرجعية لمنصة التواصل الاجتماعي

1.3.1 الدراسة الأولى

بنية بيانات وتخزين الشبكة الاجتماعية

"TAO: Facebook's Distributed Data Store for the Social Graph"

تتناول هذه الدراسة النظام TAO الذي طوّره شركة Facebook لإدارة العلاقات بين المستخدمين (Social Graph) بكفاءة عالية، يهدف النظام إلى تحسين سرعة الاستعلام والوصول إلى البيانات من خلال Data Store مخصص ومُحسّن للقراءة، مع واجهة API موحّدة تسهّل عمليات الاستعلام دون الحاجة لفهم البنية الداخلية للبيانات ويُعتبر TAO نموذجًا لتصميم قاعدة بيانات مخصّصة للتطبيقات الاجتماعية ذات الحجم الكبير، حيث يجمع بين MySQL و Caching Layer لتقليل زمن الاستجابة وتحسين الأداء.

أبرز الدروس المستفادة:

- تصميم طبقة بيانات متخصصة لتتناسب طبيعة التطبيقات الاجتماعية.
- استخدام Caching و API Layer يقلّل من الحمل على قواعد البيانات.
- التوزيع الجغرافي للبيانات يُحسّن من سرعة الوصول للمستخدمين حول العالم.

المصدر: Nathan Bronson et al., USENIX ATC, 2013 – Facebook Research.

1.3.2 الدراسة الثانية

بنية الأنظمة وتحجيم منصات التراسل/التواصل

"Scaling big data mining infrastructure: the Twitter experience"

تستعرض هذه الدراسة تجربة Twitter في بناء بنية تحتية قادرة على التعامل مع مليارات الرسائل يوميًا مع الحفاظ على الأداء اللحظي (Real-time). استخدمت Twitter تقنيات مثل Kafka, HBase, Distributed Storage Systems لمعالجة التدفق المستمر من البيانات وتحليلها بسرعة. تركّز التجربة على أهمية بناء نظام مرّن (Scalable) يسمح بتحديث المكونات دون توقف الخدمة، وتحقيق توازن بين Latency و Throughput لتقديم تجربة تفاعلية مستقرة. أبرز الدروس المستفادة:

- اعتماد تقنيات Streaming و Message Queue مثل Kafka ضروري للميزات اللحظية.
- بناء بنية مرنة قابلة للتوسع والصيانة دون انقطاع.
- أهمية أدوات المراقبة (Monitoring) والـ Logging في تحليل الأداء.

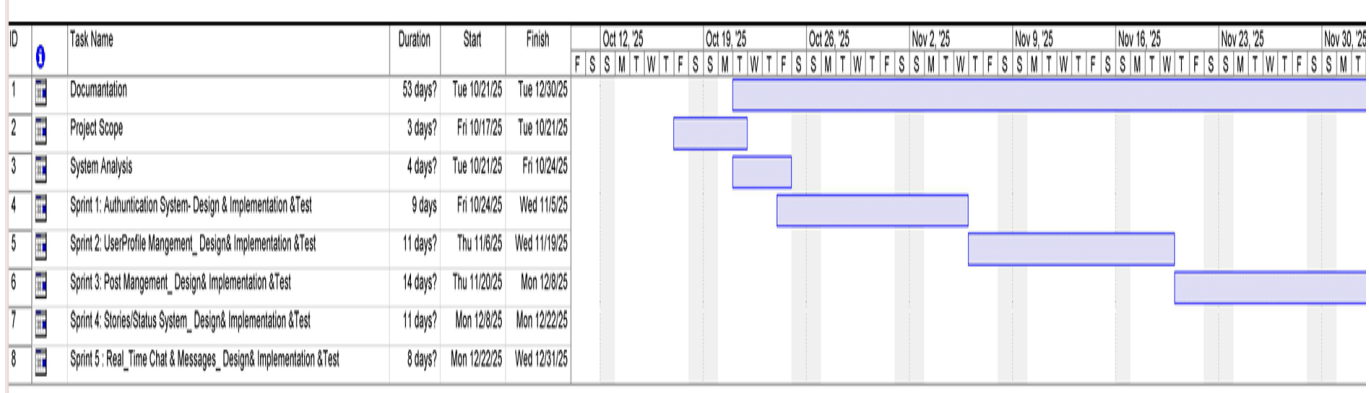
المصدر: Twitter Engineering Reports, Scaling Big Data Systems.

1.3.3 جدول مقارنة بين الدراسات المرجعية.

جدول 1.1 : مقارنة بين الدراسات المرجعية

الدراسة	TAO–Facebook	Twitter	مشروعنا NOONIFY
التقنيات المستخدمة	Data Store Caching Layer,API	Kafka,HBase,Distributed Storage.	MERN Stack
الإيجابيات	أداء عالٍ واستجابة سريعة للبيانات	معالجة بيانات ضخمة في الزمن الحقيقي	واجهات حديثة , أداء فعال تكاملاً مكونات جاهزة.
السلبيات	تعقيد البنية وصعوبة الصيانة	تكلفة تشغيل عالية وتعقيد إداري	اعتماد جزئي على خدمات خارجية.
التحديات	الحفاظ على اتساق البيانات الموزعة	التوازن بين السرعة والتوافر	ضمان الأمان والخصوصية مع الأداء العالي.
أوجه التميز	تخصيص النظام لمعالجة العلاقات الاجتماعية	بنية مرنة تدعم التحليل اللحظي	دمج تقنيات حديثة بمرونة عالية وسرعة تطوير.

1.4 الجدول الزمني Gantt Chart

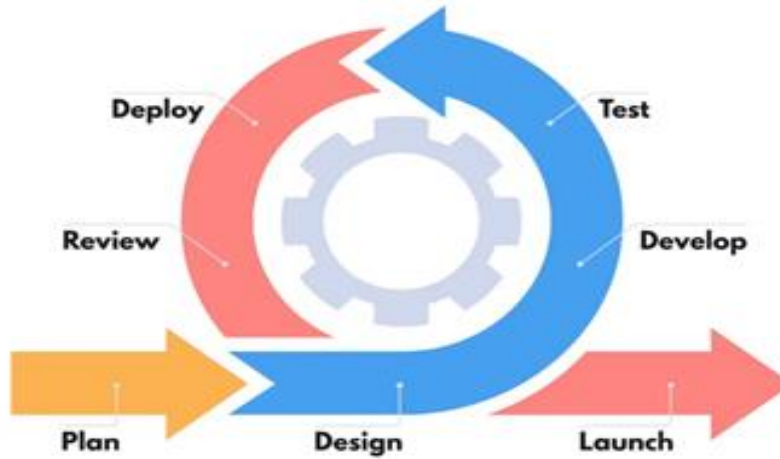


منط 1.1 : ال Gantt Chart

1.5 منهجية المتبعة في تنفيذ المشروع (منهجية SCRUM)

تم اعتماد منهجية SCRUM كإطار عمل (Agile Framework) لتنفيذ هذا المشروع، نظرًا لمرونتها العالية وقدرتها على تنظيم عملية التطوير بطريقة تفاعلية وتدرجية (Iterative and Incremental Process) , تُعد منهجية SCRUM واحدة من أكثر منهجيات Agile Software Development استخدامًا في المشاريع البرمجية الحديثة، حيث تركز على تقسيم العمل إلى مراحل قصيرة تُعرف باسم Sprints، يتم خلالها تنفيذ مجموعة محددة من المهام ذات الأولوية العالية، ومراجعتها بشكل مستمر لتحسين جودة المنتج النهائي.

SCRUM METHODOLOGY



الشكل 1.1 : ال scrum methodology

1.5.1 تطبيق منهجية SCRUM في مشروع منصة التواصل الاجتماعي

تم تقسيم عملية التطوير إلى عدة Sprints، وكل Sprint يركّز على مجموعة محددة من الوظائف الأساسية ضمن المنصة.

والوظائف هي:

Sprint 1: إعداد بيئة العمل (Setup Environment) وتنفيذ نظام التسجيل والدخول (Authentication) (System).

Sprint 2: تطوير واجهة المستخدم (Front-End Interface) باستخدام React.js وربطها مع Back-End APIs.

Sprint 3: تنفيذ ميزات البحث وإضافة الأصدقاء (Search & Friends System).

Sprint 4: تطوير نظام المنشورات والتعليقات (Posts & Comments Module).

Sprint 5: إضافة القصص (Stories System) والإشعارات (Notifications).

بعد كل Sprint، يتم مراجعة (Sprint Review Meeting) لعرض التقدم وتقييم ما تم إنجازه، يعقبه اجتماع تخطيطي (Sprint Planning Meeting) لتحديد مهام الدورة التالية.

1.6 المعمارية (System Architecture)

1.6.1 أولاً: معمارية Client-Server

تم اعتماد معمارية Client-Server كأساس لبناء المنصة، وهي من أكثر المعماريات استخداماً في تطبيقات الويب الحديثة، خصوصاً تلك التي تعتمد على الاتصال بين واجهة المستخدم (Client) والخادم (Server) عبر بروتوكول HTTP/HTTPS.

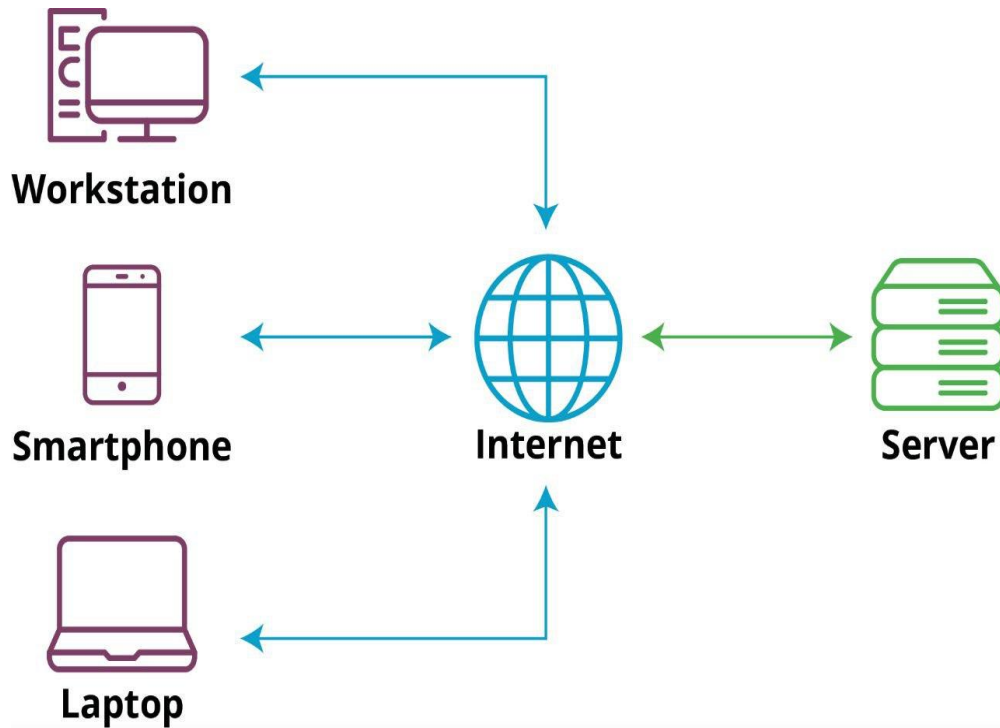
في هذا المشروع، يمثل الـ Client واجهة المستخدم المبنية باستخدام React.js، بينما يمثل الـ Server الخادم الخلفي المطور باستخدام Node.js و Express.js والمتصل بقاعدة البيانات MongoDB. يتم تبادل البيانات بين الطرفين بصيغة JSON عبر واجهات RESTful APIs.

آلية العمل:

1. يقوم المستخدم (Client) بإرسال طلب (Request) عبر المتصفح مثل تسجيل الدخول أو عرض المنشورات.
2. يستقبل الخادم (Server) الطلب من خلال Express.js Controllers ويعالجه وفقًا للمنطق البرمجي.
3. يتصل الخادم بقاعدة البيانات MongoDB لاسترجاع أو تعديل البيانات المطلوبة.
4. يعيد الخادم استجابة (Response) إلى Client، تُعرض النتيجة عبر الواجهة باستخدام React.js.

1.6.1.1 مزايا معمارية Client-Server

- فصل واضح بين الواجهة والتطبيق الخلفي (Separation of Concerns) مما يسهل التطوير والتحديث.
- أمان أعلى لأن معالجة البيانات الأساسية تتم في الخادم وليس في واجهة المستخدم.
- قابلية التوسع (Scalability) بإمكانية توزيع الخوادم أو تحديث الواجهة بشكل مستقل.
- إعادة الاستخدام (Reusability) للخدمات الخلفية عبر واجهات API يمكن استدعاؤها من تطبيقات أخرى مستقبلاً (مثل تطبيق موبايل).



الشكل 1.2 : Client_Server

1.6.2 ثانيًا: معمارية MVC (Model – View – Controller)

بالإضافة إلى معمارية Client-Server، سيتم تطبيق نمط MVC داخل الخادم (Server-Side) لتنظيم الكود وتحسين قابلية الصيانة والتوسع.

تقوم معمارية MVC على تقسيم النظام إلى ثلاث طبقات رئيسية:

1. Model (النموذج): تمثل البيانات والهياكل المرتبطة بها، وتشمل جميع العمليات الخاصة بالتخزين والاسترجاع من قاعدة البيانات.

2. View (العرض): تمثل واجهات المستخدم التي تُعرض عبر React.js في الواجهة الأمامية.

3. Controller (المتحكم): تدير التفاعل بين Model و View، وتتولى استقبال الطلبات من المستخدم

وتنفيذ المنطق اللازم ثم إرسال النتائج إلى الواجهة.

تطبيق MVC في هذا المشروع

Controllers المستخدمة:

- MessageControl: مسؤول عن إدارة الرسائل والمحادثات.
- PostControl: مسؤول عن إنشاء، تعديل، وحذف المنشورات.
- StoryControl: مسؤول عن رفع القصص (Stories) وإدارتها.
- UserControl: مسؤول عن تسجيل المستخدمين وتحديث معلوماتهم.

Models المستخدمة:

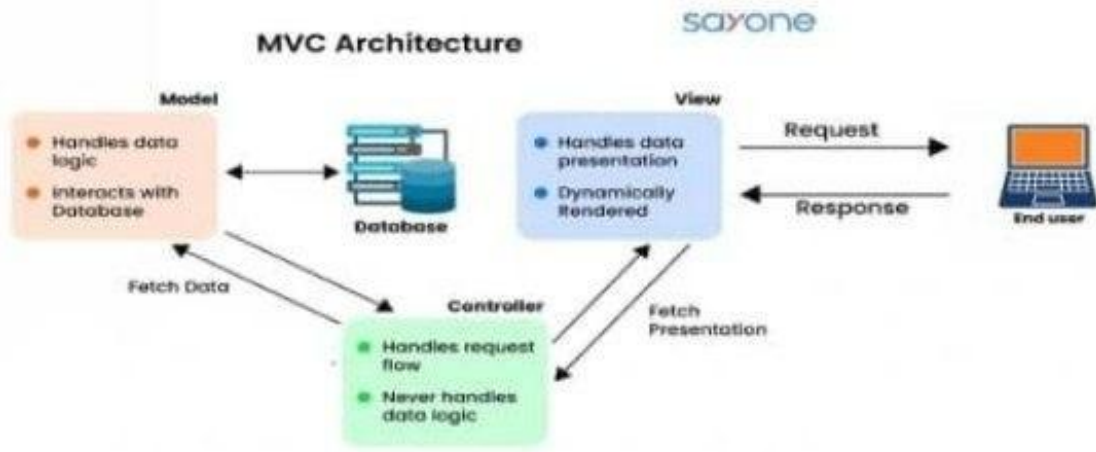
- Email, Message, Post, Story, User, Connection
- تمثل كل منها الجداول أو المجموعات (Collections) في قاعدة البيانات MongoDB، وتُعرّف الخصائص (Attributes) والعلاقات فيما بينها.

View:

- واجهات المستخدم المبنية باستخدام React.js و CSS، والتي تتلقى البيانات من الخادم عبر RESTful APIs وتعرضها للمستخدم بطريقة تفاعلية وسهلة الاستخدام.

1.6.2.1 إيجابيات معمارية MVC:

1. تنظيم الكود (Code Organization): تقسيم الكود إلى مكونات منفصلة يسهل قراءتها وصيانتها.
2. سهولة التطوير الجماعي: يمكن لفريق الواجهة الأمامية (Front-End) وفريق الخادم (Back-End) العمل بالتوازي دون تعارض.
3. إعادة الاستخدام (Reusability): يمكن إعادة استخدام الموديلات (Models) في خدمات أو مشاريع أخرى.
4. مرونة التطوير: يسهل تحديث أي جزء من النظام دون التأثير على الأجزاء الأخرى.
5. قابلية الاختبار (Testability): سهولة اختبار كل طبقة على حدة (Unit Testing).



الشكل 1.3 : mvc

1.6.3 ثالثاً: الربط بين Client_Server و MVC في المشروع :

يُعتبر النظام في هذا المشروع مزيجاً من المماريتين:

من الخارج، النظام قائم على Client-Server Architecture، حيث يتفاعل المستخدم عبر الواجهة (Client) مع الخادم (Server) عبر الإنترنت. ومن الداخل، يُدار الخادم وفق MVC Pattern الذي ينظم سير البيانات داخل النظام بين Model – Controller – View، مما يجعل النظام قابلاً للتوسع وسهل الصيانة.

Controller Layer: تستقبل الطلبات وتنفذ المنطق المطلوب من خلال وحدات مثل MessageControl وPostControl وStoryControl وUserControl.

Model Layer: تتعامل مع البيانات الممثلة في نماذج مثل User وPost وStory وMessage وConnection وEmail، وتتصل بقاعدة البيانات MongoDB.

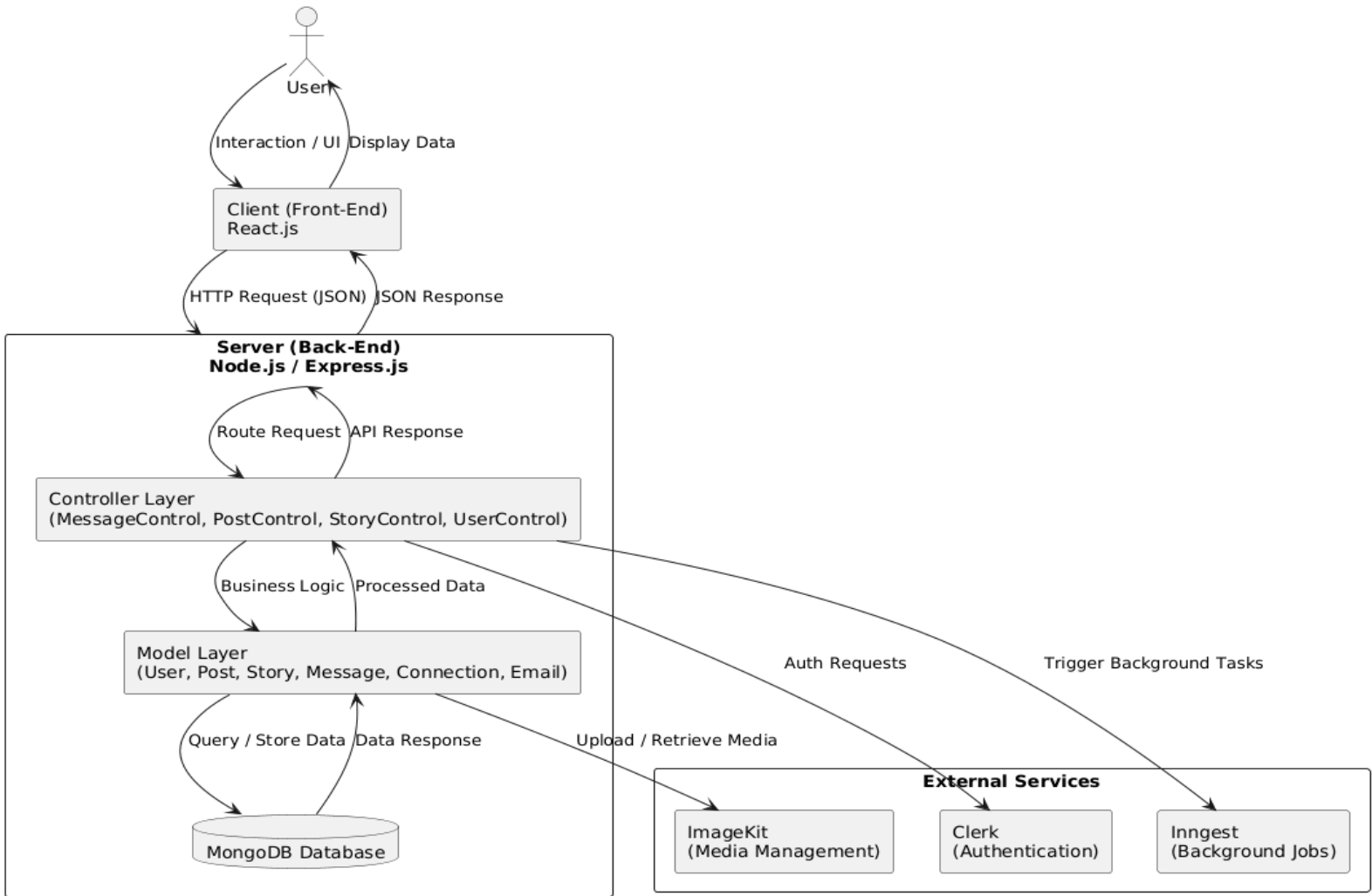
View Layer: تُرسل البيانات النهائية إلى الواجهة الأمامية ليتم عرضها للمستخدم.

يرتبط النظام أيضاً بخدمات خارجية مثل Clerk للتوثيق، وImageKit لإدارة الوسائط، وInngest للمهام الخلفية. تسير البيانات في تسلسل واضح:

المستخدم → الواجهة → الخادم (Models و Controllers) → قاعدة البيانات → ثم تعود النتائج إلى المستخدم عبر الواجهة.

بذلك تحقق المنصة توازنًا مثاليًا بين التوزيع الشبكي (Networked Architecture) وتنظيم المكونات الداخلية (Software Structure)، ما يؤدي إلى نظام قوي، آمن، وسهل التطوير المستقبلي.

Architecture Diagram - Client Server + MVC Integration



الشكل 1.4 : دمج المعماريتين

1.7 الأدوات المستخدمة

يعرض هذا الفصل الأدوات والتقنيات التي تم الاعتماد عليها في تطوير منصة التواصل الاجتماعي **Noonify**، حيث تم تقسيم الأدوات إلى ثلاث فئات رئيسية: أدوات الواجهة الأمامية (Front-End) ، وأدوات الواجهة الخلفية (Back-End) ، وأدوات قاعدة البيانات (Database) ، وذلك بما يتوافق مع معمارية **MERN Stack**.

1.7.1 أدوات الواجهة الأمامية (Front-End Tools)

تم تطوير الواجهة الأمامية للتطبيق باستخدام مجموعة من الأدوات الحديثة التي تهدف إلى بناء واجهة مستخدم تفاعلية وسريعة الاستجابة، ومن أبرزها:

• **React.js**

مكتبة JavaScript تُستخدم لبناء واجهات المستخدم المعتمدة على المكونات (Component-Based Architecture)، مما يسهل إعادة استخدام الكود وتحسين قابلية الصيانة.

• **Vite**

أداة بناء (Build Tool) حديثة تُستخدم لتسريع عملية التطوير وتشغيل المشروع بكفاءة أعلى مقارنة بالأدوات التقليدية.

• **JavaScript (ES6+)**

لغة البرمجة الأساسية المستخدمة في تطوير منطق الواجهة الأمامية والتعامل مع الأحداث والتفاعل مع المستخدم.

• HTML5

تُستخدم لبناء الهيكل العام لصفحات الويب.

• CSS3

تُستخدم لتنسيق واجهات المستخدم وتحسين التجربة البصرية.

• Axios

مكتبة تُستخدم لإرسال واستقبال طلبات HTTP بين الواجهة الأمامية والواجهة الخلفية.

• ESLint

أداة لتحليل الكود واكتشاف الأخطاء البرمجية وتحسين جودة الكود.

• Vercel

منصة تُستخدم لنشر تطبيق الواجهة الأمامية وتشغيله على بيئة إنتاجية.

1.7.2 أدوات الواجهة الخلفية (Back-End Tools)

تم بناء الواجهة الخلفية للتطبيق باستخدام بيئة تشغيل وتقنيات تضمن الأداء العالي وتنظيم منطق النظام، ومن أهم هذه الأدوات:

• Node.js

بيئة تشغيل تعتمد على JavaScript تُستخدم لتنفيذ كود الخادم (Server-Side) بكفاءة عالية.

• Express.js

إطار عمل (Framework) مبني على Node.js يُستخدم لإنشاء واجهات برمجية (RESTful APIs) وتنظيم المسارات (Routes).

• JavaScript

تُستخدم كلغة أساسية لتطوير منطق الخادم.

• JWT (JSON Web Token)

تُستخدم لتنفيذ نظام المصادقة (Authentication) والتحقق من هوية المستخدم.

• Multer

مكتبة تُستخدم لمعالجة ورفع الملفات (مثل الصور) من المستخدم إلى الخادم.

• NodeMailer

مكتبة تُستخدم لإرسال رسائل البريد الإلكتروني، مثل رسائل التحقق أو الإشعارات.

• ImageKit

خدمة تُستخدم لإدارة الصور ورفعها وتخزينها بشكل سحابي.

• Middleware

تُستخدم لمعالجة الطلبات الوسيطة مثل التحقق من الصلاحيات وتأمين المسارات.

1.7.3 أدوات قاعدة البيانات (Database Tools)

تم استخدام قاعدة بيانات مرنة وقابلة للتوسع لتخزين بيانات المستخدمين والمحتوى، وتشمل الأدوات التالية:

• MongoDB

قاعدة بيانات NoSQL تعتمد على المستندات (Documents) وتتميز بالمرونة وسهولة التعامل مع البيانات غير المهيكلة.

• Mongoose

مكتبة تُستخدم كنظام نمذجة (ODM) للتعامل مع MongoDB ، حيث تُسهل إنشاء النماذج (Schemas) وتنفيذ العمليات على البيانات.

1.7.4 ملخص الأدوات المستخدمة

اعتمد المشروع على **MERN Stack** كمعمارية رئيسية، حيث تم تحقيق التكامل بين الواجهة الأمامية، الواجهة الخلفية، وقاعدة البيانات، مما ساهم في بناء منصة تواصل اجتماعي حديثة، منظمة، وقابلة للتطوير، وتلبي المتطلبات الأكاديمية والعملية للمشروع.

2 الفصل الثاني : الدراسة التحليلية

2.1 المتطلبات العامة (General Requirements)

2.1.1 نظام التسجيل والدخول (Authentication System)

1. إمكانية إنشاء حساب جديد (Sign Up) باستخدام البريد الإلكتروني أو رقم الهاتف.
2. تسجيل الدخول (Sign In) باستخدام بيانات الاعتماد أو عبر Social Login (Google / GitHub).
3. تفعيل خاصية إعادة تعيين كلمة المرور (Reset Password) عند نسيانها.
4. تسجيل الخروج من الجلسة الحالية (Log Out).

2.1.2 إدارة الملف الشخصي (User Profile Management)

1. عرض المعلومات الأساسية للمستخدم (الاسم، الصورة، Bio، عدد المتابعين).
2. تعديل الملف الشخصي (Edit Profile) وتحديث البيانات.
3. عرض ملفات المستخدمين الآخرين (View Other Profiles).
4. عرض حالة الاتصال (Online / Offline Status).
5. حماية الدخول وإدارة الجلسات باستخدام Tokens و Clerk Authentication.

2.1.3 إدارة المنشورات (Posts Management)

1. إنشاء منشورات جديدة (Create Post) نصية أو تحتوي على صور.
2. تعديل أو حذف المنشورات (Edit / Delete Post).
3. عرض المنشورات في الصفحة الرئيسية (Feed) بترتيب زمني.
4. التفاعل مع المنشورات عبر الإعجابات (Like) والتعليقات (Comment).
5. حفظ المنشورات المفضلة (Save Post) أو مشاركتها (Share Post).
6. الإبلاغ عن المنشورات المخالفة (Report Post).

2.1.4 إدارة القصص (Stories / Status System)

1. رفع قصة جديدة (Upload Story) بصيغة صورة أو فيديو قصير.
2. عرض القصص الخاصة بالمستخدمين الذين يتابعهم (View Stories).
3. حذف القصة يدويًا أو اختفاؤها تلقائيًا بعد 24 ساعة.
4. عرض قائمة المشاهدين (Viewers List).

2.1.5 نظام المراسلة (Real-time Chat & Messages)

1. بدء محادثة جديدة (Start Conversation) بين المستخدمين.
2. إرسال واستقبال الرسائل النصية والصورية والرموز التعبيرية (Emojis).
3. دعم المراسلة اللحظية (Real-time Chat) باستخدام Socket.io.

4. عرض حالة الرسائل (مرسلة - مستلمة - مقروءة).

5. حذف أو كتم المحادثة (Mute / Delete Chat).

2.1.6 نظام العلاقات الاجتماعية (Connections System)

1. متابعة أو إلغاء متابعة المستخدمين (Follow / Unfollow).

2. إرسال واستقبال طلبات الصداقة (Friend Requests).

3. عرض قوائم المتابعين والمتابعين (Followers / Following Lists).

4. اقتراح أصدقاء بناءً على الاهتمامات المشتركة (Friend Suggestions).

5. حظر أو إلغاء حظر المستخدمين (Block / Unblock).

2.1.7 نظام البحث والاكتشاف (Search & Discover System)

1. البحث عن المستخدمين بالاسم أو اسم المستخدم (Username Search).

2. البحث عن المستخدمين من خلال الموقع.

3. البحث عن المنشورات باستخدام الوسوم (Hashtags).

4. عرض صفحة اكتشاف (Discover Page) تحتوي على المنشورات الأكثر تفاعلاً والمستخدمين

المقترحين.

2.1.8 نظام الإشعارات (Notifications System)

1. إرسال إشعارات فورية (Real-time Notifications) عند التفاعل مع المنشورات أو المتابعة

الجديدة.

2. عرض جميع الإشعارات في صفحة مخصصة (Notifications Page).
3. إمكانية حذف الإشعارات أو وضع علامة "تمت القراءة" (Mark as Read).

2.1.9 إدارة الوسائط (Media Management)

- 3 رفع الصور عبر ImageKit وتحسينها تلقائيًا.
- 4 دعم صيغ الصور المختلفة (JPEG, PNG, WEBP).
- 5 ضغط الصور لتقليل الحجم دون فقدان الجودة.
- 6 حماية روابط الوسائط من الوصول غير المصرح به (Access Control).

6.1.1 النظام الخلفي (Backend Operations)

1. إنشاء واجهات للتفاعل مع المستخدمين والمنشورات والرسائل.
2. تنفيذ المهام المجدولة عبر Inngest مثل حذف القصص المنتهية.
3. تسجيل الأخطاء والنشاطات (Error Logging & Monitoring).

2.2 المتطلبات غير الوظيفية (Non-Functional)

(Requirements)

1. Security: استخدام Clerk للتحقق.
2. Performance: سرعة تحميل عالية، استجابة فورية، Caching، Real-time Chat.
3. Scalability: دعم التوسعة الأفقية باستخدام Node.js و MongoDB.

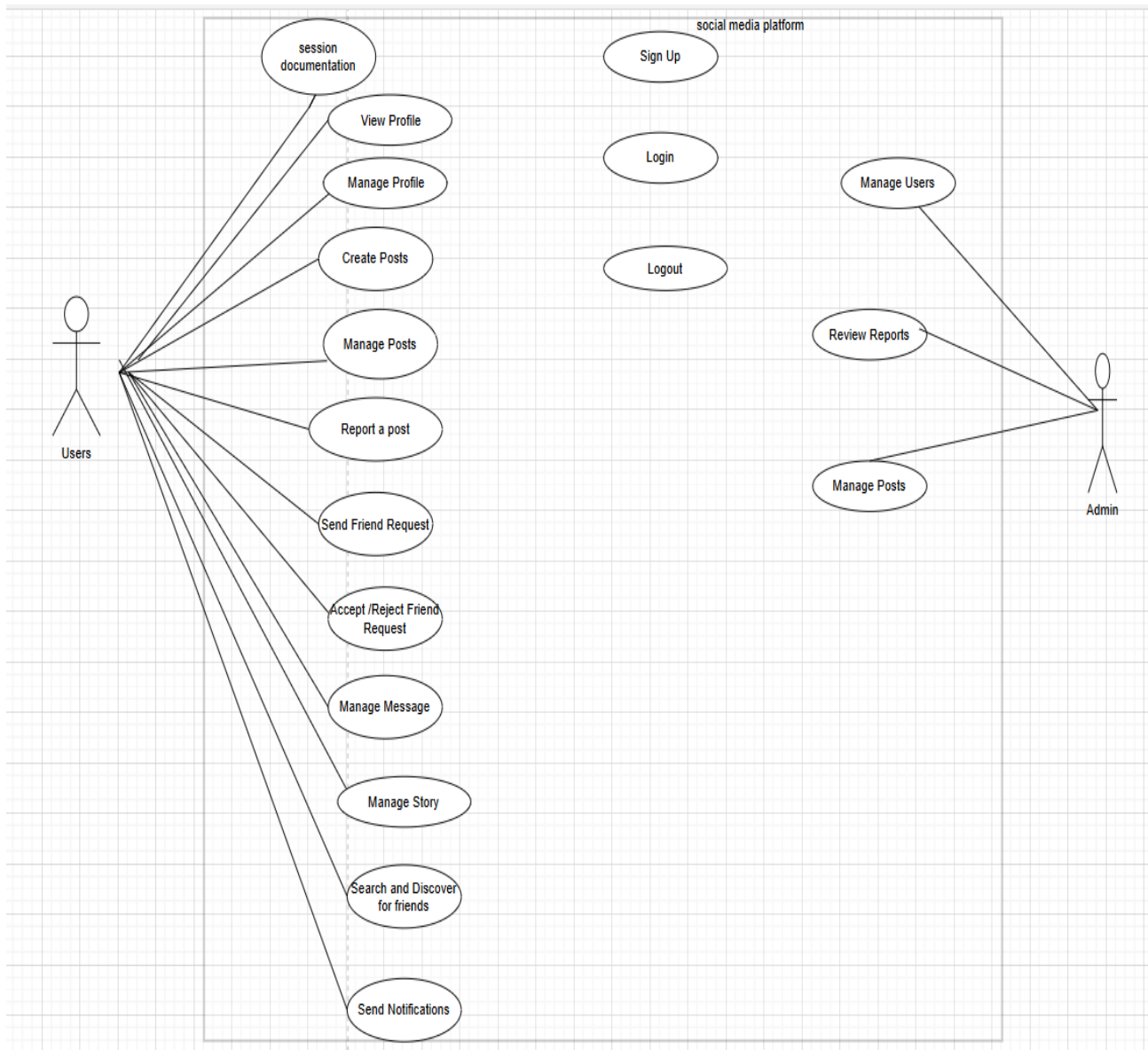
4. Usability: واجهة تفاعلية، تصميم متجاوب (Responsive Design).
5. Reliability: تشغيل مستقر، مراقبة النظام (Monitoring & Logging).
6. Maintainability: كود منظم (Modular Code)، توثيق جيد.
7. Compatibility: دعم جميع المتصفحات والأجهزة (Cross-Platform).
8. Privacy: حماية بيانات المستخدمين، إعدادات خصوصية مرنة.

2.3 المتطلبات التي سيتم تنفيذها في المشروع الفصلي

1. إمكانية إنشاء حساب جديد (Sign Up) باستخدام البريد الإلكتروني أو رقم الهاتف.
2. تسجيل الدخول (Sign In) باستخدام بيانات الاعتماد أو عبر Social Login (Google / GitHub).
3. التحقق من تسجيل الدخول من خلال رمز التحقق.
4. تسجيل الخروج من الجلسة الحالية (Log Out).
5. عرض المعلومات الأساسية للمستخدم (الاسم، الصورة، Bio، عدد المتابعين).
6. تعديل الملف الشخصي (Edit Profile) وتحديث البيانات.
7. عرض ملفات المستخدمين الآخرين (View Other Profiles).
8. البحث عن المستخدم من خلال الاسم أو البايو أو الموقع.
9. إنشاء منشورات جديدة (Create Post) نصية أو تحتوي على صور.
10. تعديل أو حذف المنشورات (Edit / Delete Post).

11. عرض المنشورات في الصفحة الرئيسية (Feed) بترتيب زمني.
12. التفاعل مع المنشورات عبر الإعجابات (Like)
13. التفاعل مع المنشورات عبر التعليقات (Comment).
14. مشاركة المنشورات (Share Post).
15. عرض حالة الطلب .
16. رفع قصة جديدة (Upload Story) بصيغة صورة أو فيديو قصير.
17. عرض القصص الخاصة بالمستخدمين الذين يتابعهم (View Stories).
18. عرض قائمة المشاهدين (Viewers List).
19. Security: استخدام Clerk للتحقق، و HTTPS لتشفير البيانات.
20. Usability: واجهة تفاعلية، تصميم متجاوب (Responsive Design).
21. Compatibility: دعم جميع المتصفحات والأجهزة (Cross-Platform).

2.4 مخطط ال Use Case Diagram



الشكل 2.5 : ال Class Diagram

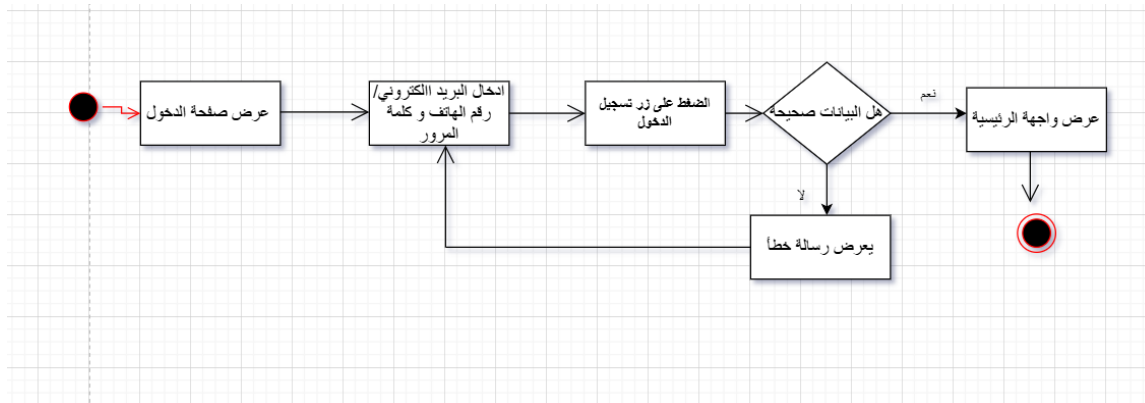
2.5 توصيف حالات الاستخدام Use Case Specifications

2.6 مخططات النشاط Activity Diagrams

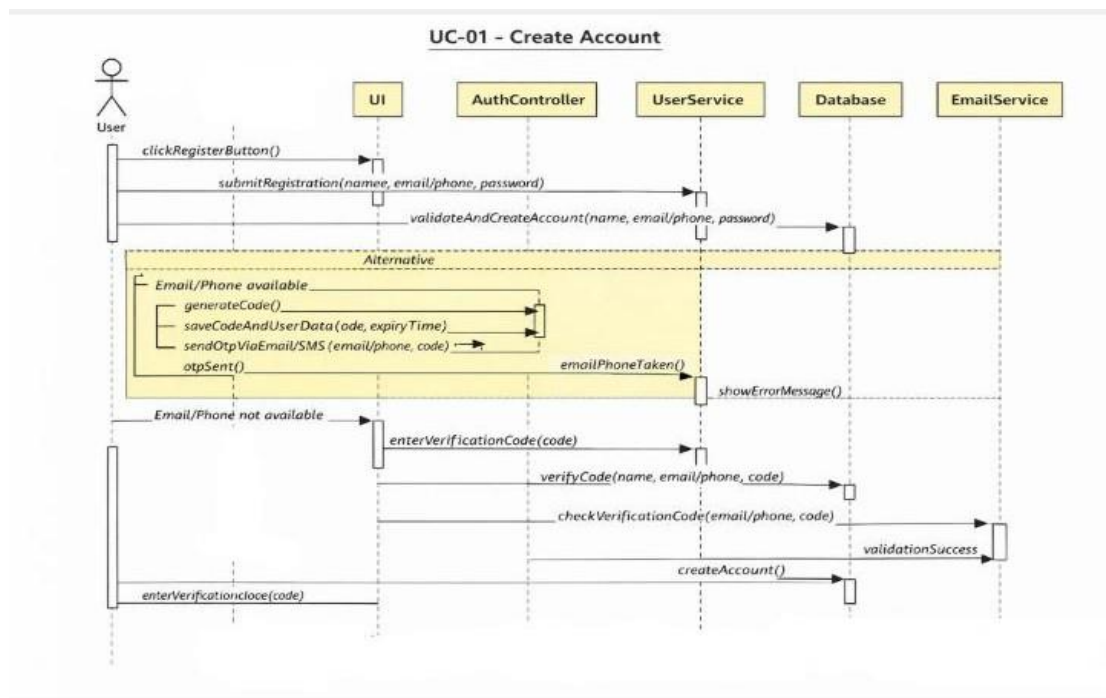
2.7 مخططات التسلسل Sequence Diagrams

جدول 2.2 : توصيف حالة تسجيل الحساب

Register	UC-01	Use Case Name
المستخدم		Actors
تمكين المستخدم من انشاء حساب جديد على النظام باستخدام البريد الالكتروني او رقم الهاتف للتحقق من الهوية والوصول الى خدمات النظام.		Description
ليس لديه أي تسجيل دخول.		Precondition
1- يدخل المستخدم التطبيق . 2- يعرض النظام نموذج التسجيل . 3- يدخل المستخدم البيانات (الاسم و البريد الالكتروني وكلمة المرور او عن طريق GOOGLE) . 4- يضغط المستخدم على زر "REGISTER" . 5- يتحقق النظام من صحة البيانات ويرسل رمز تحقق لينشئ الحساب. 6- يدخل المستخدم رمز التحقق الذي ارسل له عبر بريده الالكتروني. 7- يرسل رسالة تأكيد ويوجه المستخدم.		Main flow
A1: المستخدم يلغي العملية ويعود للحالة السابقة. A2: المستخدم يدخل البريد الالكتروني مكرر يرسل رسالة البريد الالكتروني مأخوذ مسبقا. A3: لا يرسل رمز للتحقق ويقوم المستخدم بإعادة ارسال رسالة		Alternative flow
تم انشاء الحساب وتسجيل الدخول تلقائي.		Postconditions



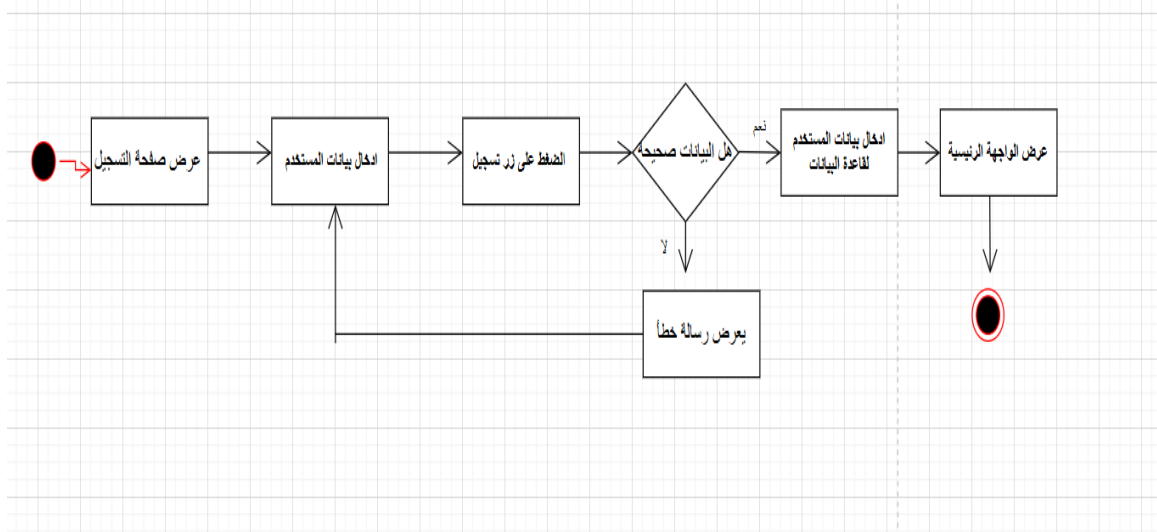
مخطط 2.2 : النشاط حالة تسجيل الحساب



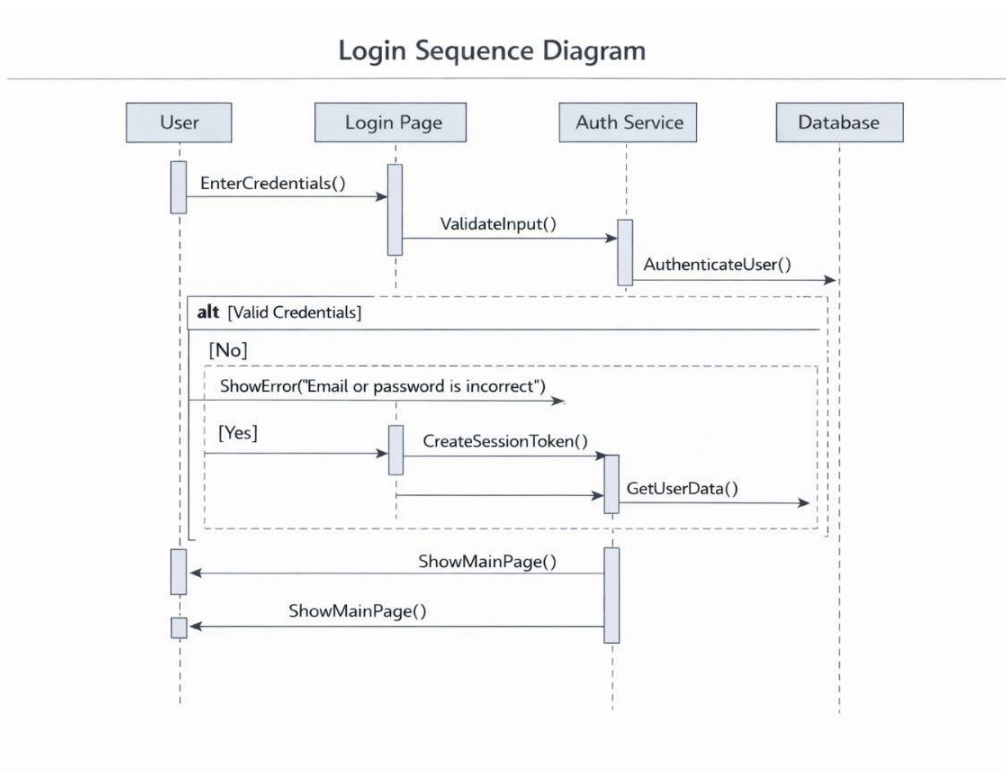
مخطط 2.3 : التسلسل حالة تسجيل الحساب

جدول 2.3 : توصيف حالة تسجيل الدخول

Login	UC-02	Use Case Name
	المستخدم	Actors
	يتيح للمستخدم الوصول الى حسابه المسجل.	Description
	ليس لديه أي تسجيل دخول.	Precondition
	1- يفتح المستخدم صفحة تسجيل الدخول . 2- يدخل المستخدم البريد الإلكتروني أو رقم الهاتف وكلمة المرور. 3- يتحقق النظام من صحة البيانات مقابل قاعدة البيانات أو خدمة Clerk Authentication. 4- إذا كانت البيانات صحيحة النظام يصدر Token للجلسة ويعطي المستخدم وصولاً كاملاً للنظام. 5- النظام يعرض واجهة المستخدم الرئيسية بعد تسجيل الدخول.	Main flow
	A1: facebook ,google. يتيح النظام تسجيل دخول عبر حساب A3: يعرض النظام رسالة خطأ "البريد الإلكتروني او كلمة المرور غير صحيحة"	Alternative flow
	يتم التحقق من هوية المستخدم ويسمح للمستخدم بالوصول الى حسابه وميزات النظام المختلفة.	Postconditions



مخطط 2.4 : النشاط حالة تسجيل الدخول



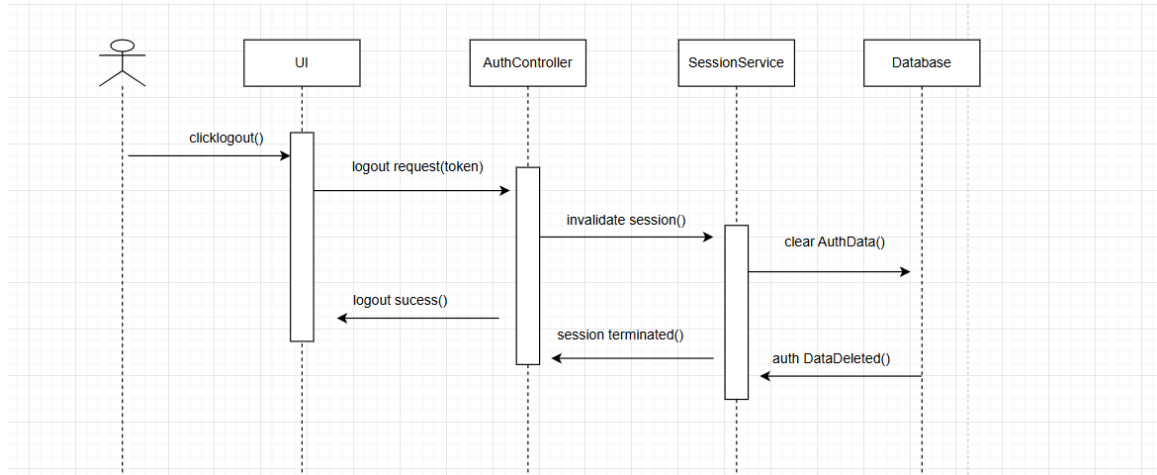
مخطط 2.5 : التسلسل حالة تسجيل الدخول

جدول 2.4: توصيف حالة الدخول الآمن

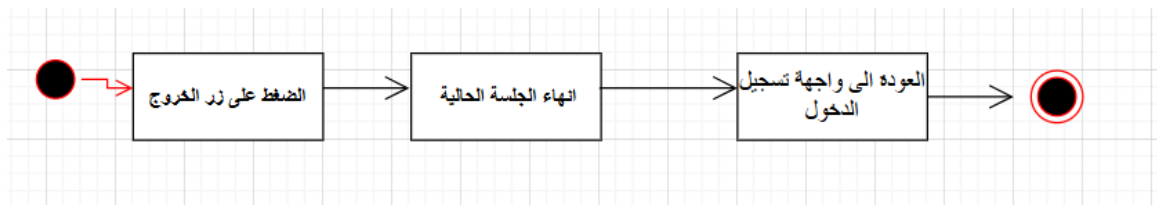
Login Authentication	UC_03	Use Case Name
المستخدم		Actors
يُتيح للمستخدم اكمال عمالية تسجيل الدخول بعد ادخال اسم المستخدم وكلمة المرور بنجاح من خلال رمز التحقق المؤقت, يتم ارساله الى البريد الالكتروني لتعزيز الأمان.		Description
المستخدم قام بإدخال اسم المستخدم وكلمة المرور بشكل صحيح.		Precondition
1- النظام يتحقق من صحة اسم المستخدم وكلمة المرور. 2- النظام ينشئ رمز تحقق مؤقت صالح لمدة زمنية محددة ليريد الالكتروني للمستخدم. 3- النظام يعرض شاشة ادخال الرمز ليقوم المستخدم بإدخاله. 4- النظام يتحقق من صحة الرمز المدخل ومن صلاحيته الزمنية. 5- النظام يؤكد نجاح العملية وينشئ الجلسة ويقوم بتوجيه المستخدم للصفحة الرئيسية.		Main flow
المستخدم يدخل رمز تحقق خاطئ النظام يعرض رسالة خطأ توضح ان: A2: الرمز غير صحيح.. انتهاء صلاحية رمز التحقق, النظام يعرض رسالة بانتهاء صلاحية الرمز: A4: حيث يوفر خيار إعادة رمز جديد.		Alternative flow
تم انشاء الجلسة بأمان.		Postconditions

جدول 2.5: توصيف حالة تسجيل الخروج

Logout	UC_04	Use Case Name
المستخدم		Actors
انهاء الجلسة الحالية والخروج بأمان من الحساب.		Description
النظام مسجل دخول بالفعل الى النظام.		Precondition
1- المستخدم يضغط على زر تسجيل الخروج. 2- النظام يقوم بإلغاء الجلسة الحالية ويحذف بيانات المصادقة من المتصفح. 3- النظام يعيد توجيه المستخدم الى صفحة تسجيل الدخول ويعرض رسالة "تم تسجيل الخروج بنجاح".		Main flow
A1: المستخدم يختار الغاء عملية الخروج ويبقى غي الصفحة الحالية. A4: النظام يكتشف ان الجلسة منتهية ويتم تحويله مباشرة الى تسجيل الدخول.		Alternative flow
يتم تسجيل الخروج بنجاح ويتم تحويل المستخدم الى صفحة الدخول.		Postconditions



مخطط 2.6 : التسلسل حالة تسجيل الخروج

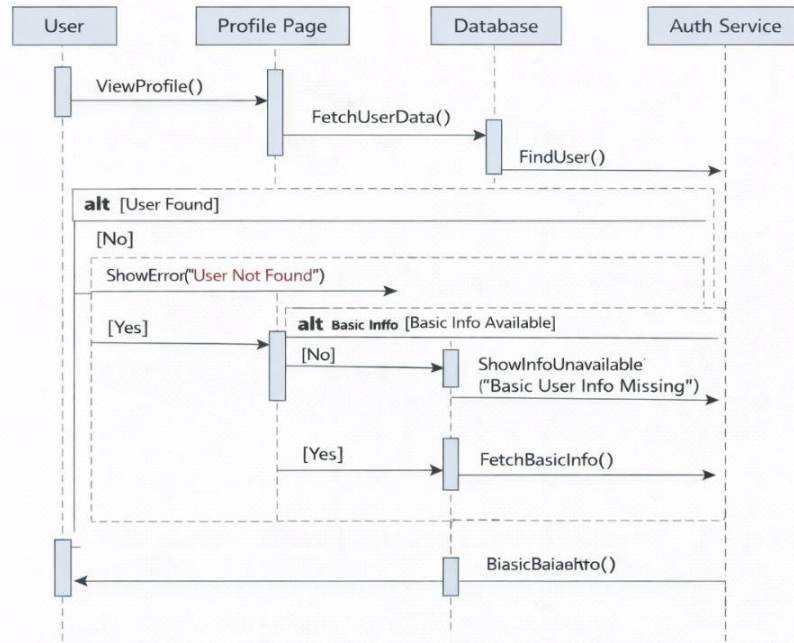


مخطط 2.7 : النشاط حالة تسجيل الخروج

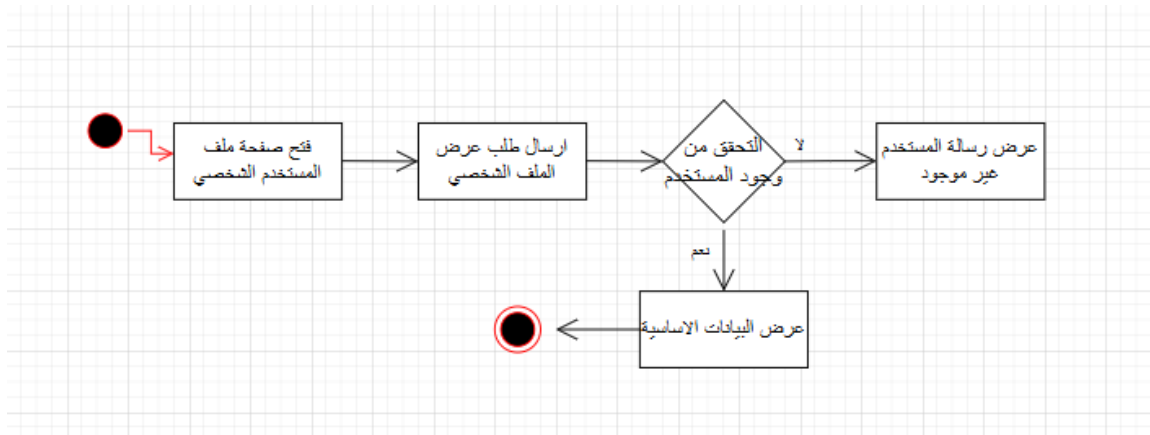
جدول 2.6 : توصيف حالة عرض الواجهة الأساسية

Display Information	UC_05	Use Case Name
المستخدم		Actors
تمكين المستخدم من رؤية المعلومات الأساسية لحساب مستخدم معين مثل الاسم والصورة الشخصية والنبذة وعدد المتابعين.		Description
1. تسجيل دخول المستخدم إلى حسابه مسبقاً.		Precondition
1- يفتح صفحة ملف المستخدم الشخصي . 2- يتحقق من وجود المستخدم في قاعدة البيانات . 3- يجلب البيانات الأساسية (الاسم, الصورة, النبذة, عدد المتابعين) . 4- يعرض المعلومات على واجهة المستخدم في صفحة ال profile.		Main flow
إذا لم يعثر على المستخدم في قاعدة البيانات يعرض النظام رسالة خطأ: A2: "المستخدم غير موجود". إذا لم تتوفر بعض المعلومات يعرض النظام عدم وجود المعلومات في الحقل: A3: الاتي .		Alternative flow
تظهر المعلومات الأساسية للمستخدم بنجاح على الشاشة .		Postconditions

Show Basic Information Sequence Diagram



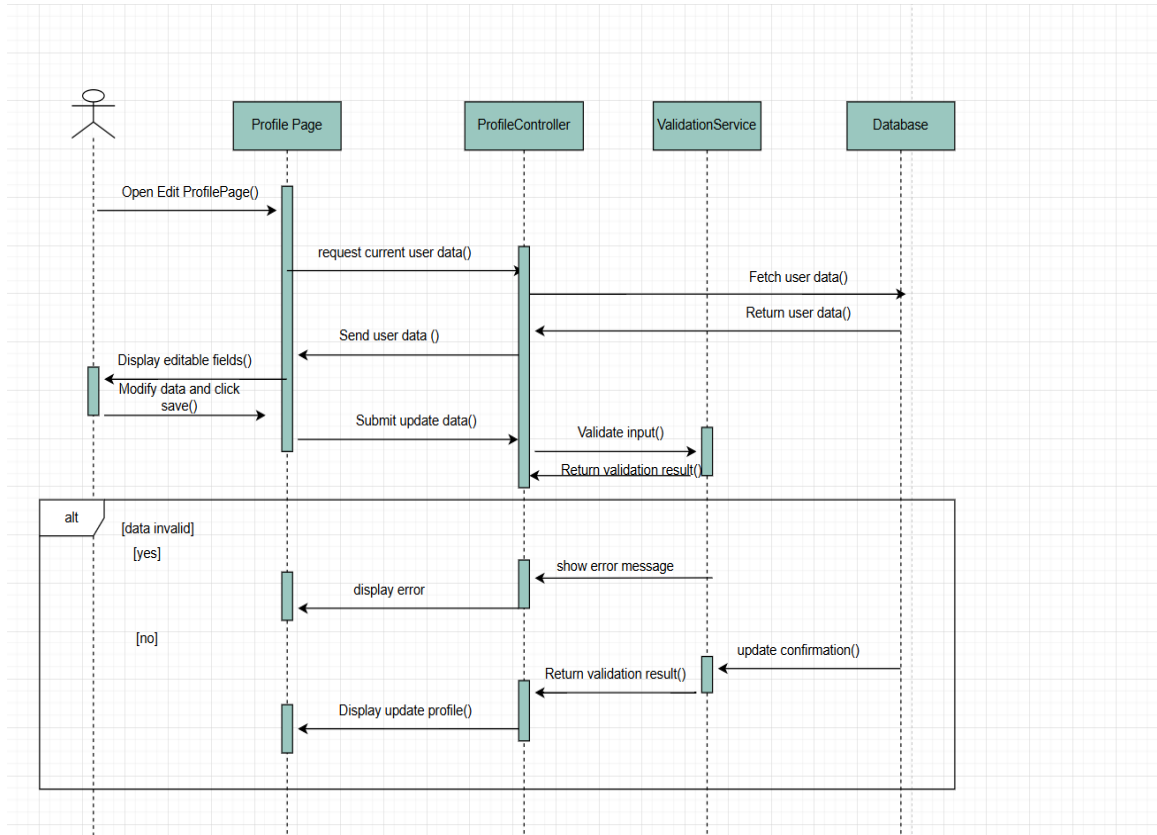
مخطط 2.8 : التسلسل حالة عرض الواجهة الأساسية



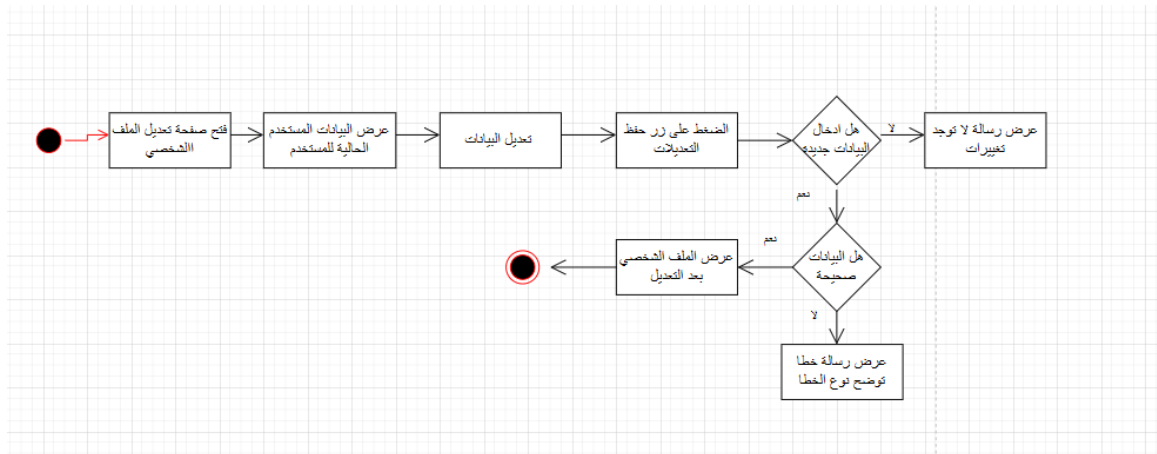
مخطط 2.9 : النشاط حالة عرض الواجهة الأساسية

جدول 2.7: توصيف حالة التعديل على الحساب

Edit Information	UC_06	Use Case Name
المستخدم		Actors
تمكين المستخدم من تعديل بياناتهم الشخصية المسجلة في النظام مثل الاسم و النبذة والصورة و البريد الالكتروني و كلمة المرور .		Description
1.تسجيل دخول المستخدم إلى حسابه مسبقا.		Precondition
1- يفتح صفحة تعديل الملف الشخصي . 2- يعرض النظام البيانات الحالية للمستخدم داخل حقول قابلة للتعديل . 3- يقوم المستخدم بتعديل البيانات المطلوبة (الاسم والصورة والبريد ...) . 4- يضغط المستخدم على زر "حفظ التعديلات" . 5- يتحقق النظام من صحة المدخلات (تنسيق البريد الالكتروني او حجم الصورة). 6- يقوم النظام بتحديث البيانات في قاعدة البيانات. 7- يعرض النظام رسالة "تم تعديل الملف الشخصي بنجاح" . 8- يعرض النظام الملف الشخصي بالبيانات الجديدة .		Main flow
A2: اذا كانت البيانات المدخلة غير صحيحة يعرض النظام رسالة خطأ توضح نوع الخطأ ويطلب من المستخدم التصحيح . A3: اذا لم يغير المستخدم أي بيانات وضغط "حفظ" يعرض النظام رسالة "لا توجد تغييرات لتحديثها" .		Alternative flow
تظهر رسالة تأكيد بنجاح عملية التحديث وتعرض البيانات الجديدة في صفحة الملف الشخصي بعد التعديل .		Postconditions



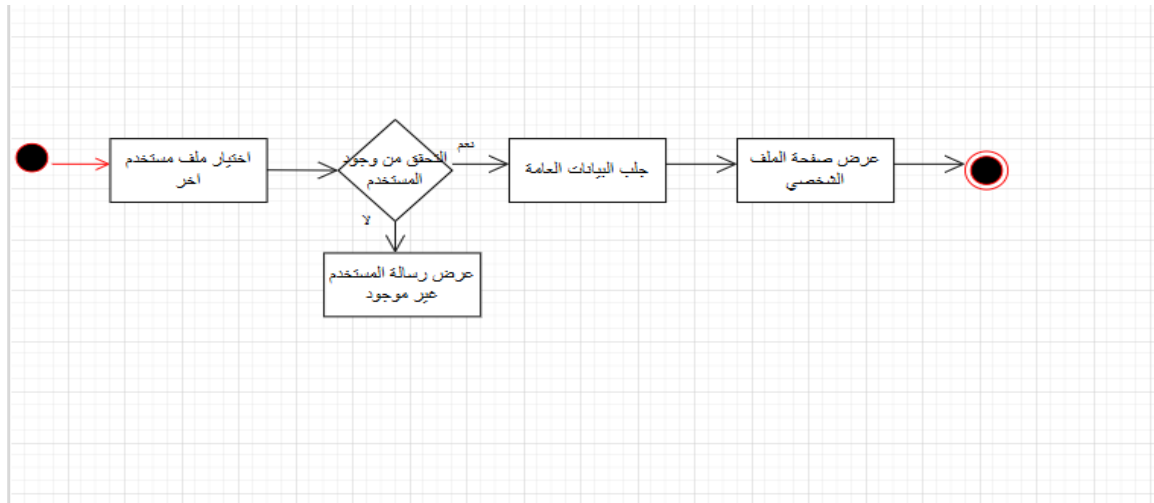
مخطط 2.10 : التسلسل حالة التعديل على الحساب



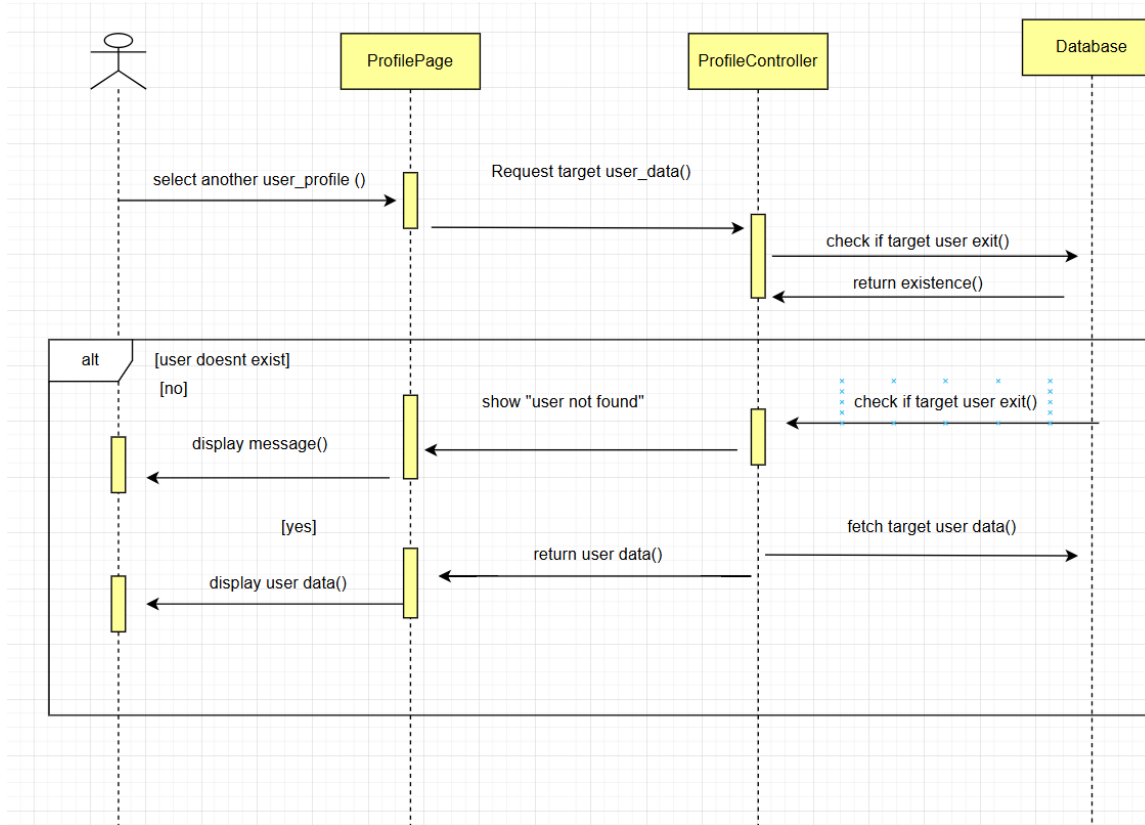
مخطط 2.11 : النشاط حالة التعديل على الحساب

جدول 2.8: توصيف حالة عرض معلومات الحسابات

Visit user page	UC_07	Use Case Name
المستخدم		Actors
تمكين المستخدم من زيارة صفحات المستخدمين الآخرين في النظام وعرض المعلومات عنه .		Description
1.تسجيل دخول المستخدم إلى حسابه مسبقا.		Precondition
1- يختار المستخدم ملف مستخدم اخر لعرضه اما من نتائج البحث او من قائمة المتابعين . 2- يتحقق النظام من وجود المستخدم الهدف في قاعدة البيانات. 3- يجلب النظام بيانات المستخدم الهدف وفق مستوى الصلاحية (الاسم والصورة والنبرة و عدد المتابعين و المنشورات العامة). 4- يعرض النظام صفحة الملف الشخصي للمستخدم الاخر بالمعلومات المسموح عرضها .		Main flow
اذا لم يجد النظام المستخدم في قاعدة البيانات يعرض النظام رسالة A2: "المستخدم غير موجود". اذا كان الملف خاصا والمستخدم الحالي ليس من المتابعين المصرح لهم A3: يعرض النظام رسالة "هذا الحساب خاص" ويظهر فقط المعلومات المحدودة (الصورة والاسم).		Alternative flow
تعرض صفحة المستخدم ملف المستخدم بالمعلومات العامة المسموح عرضها.		Postconditions



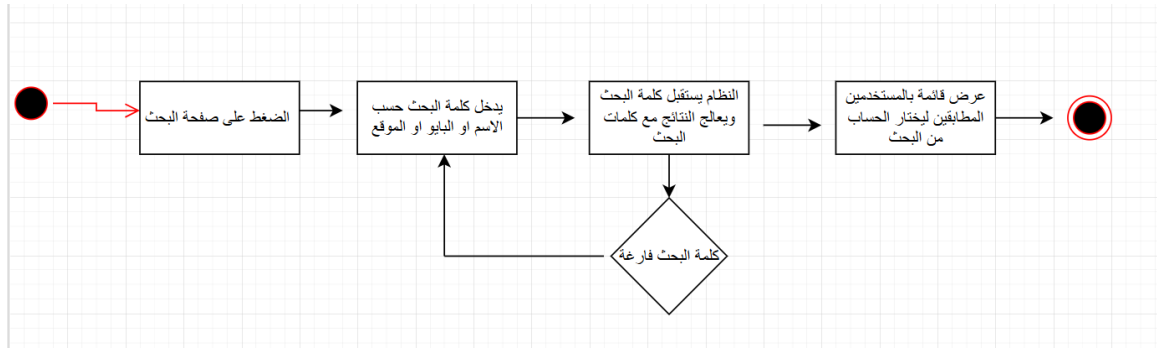
مخطط 2.12 : النشاط حالة عرض معلومات الحسابات



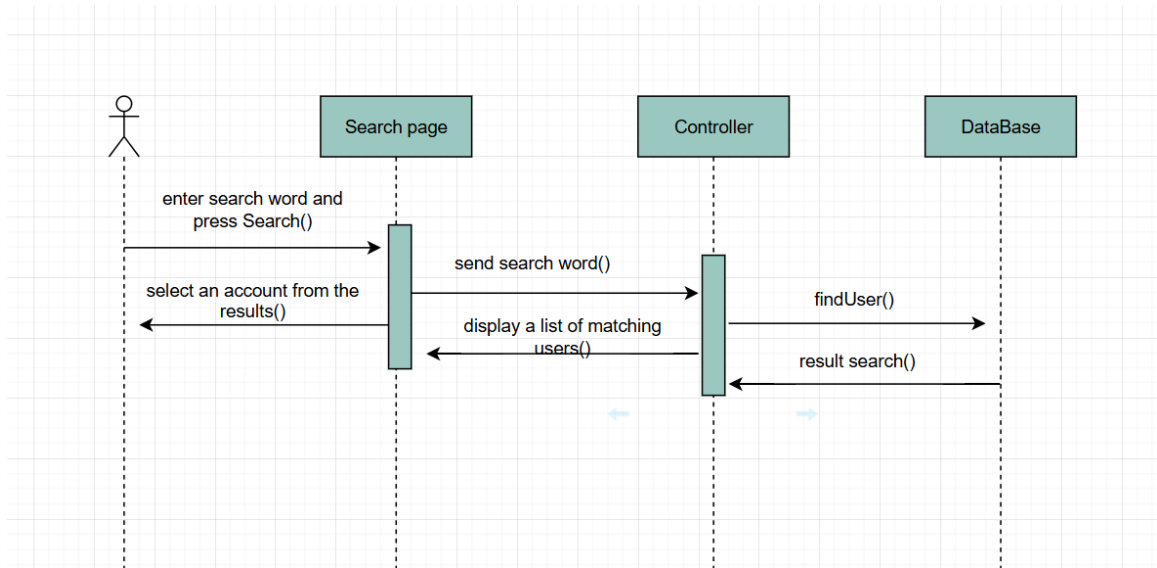
مخطط 2.13 : النشاط حالة عرض معلومات الحسابات

جدول 2.9: توصيف حالة البحث عن الأصدقاء

Search	UC_08	Use Case Name
المستخدم		Actors
يُتيح للمستخدم البحث عن مستخدمين آخرين داخل النظام باستخدام كلمات مفتاحية .		Description
1.تسجيل دخول المستخدم إلى حسابه مسبقا.		Precondition
1- يضغط المستخدم على صفحة البحث ويدخل كلمة البحث . 2- المستخدم يضغط على زر البحث . 3- النظام يستقبل كلمة البحث ويعالج النتائج ويطابقها مع كلمات البحث . 4- النظام يعرض قائمة بالمستخدمين المطابقين ويختار الحساب من نتائج البحث.		Main flow
ادخال كلمة بحث فارغة النظام يعرض رسالة تنبيه تطلب ادخال كلمة البحث. A3: لا توجد نتائج النظام يعرض رسالة لا توجد نتائج مطابقة: A4:		Alternative flow
تم عرض نتائج البحث بنجاح.		Postconditions



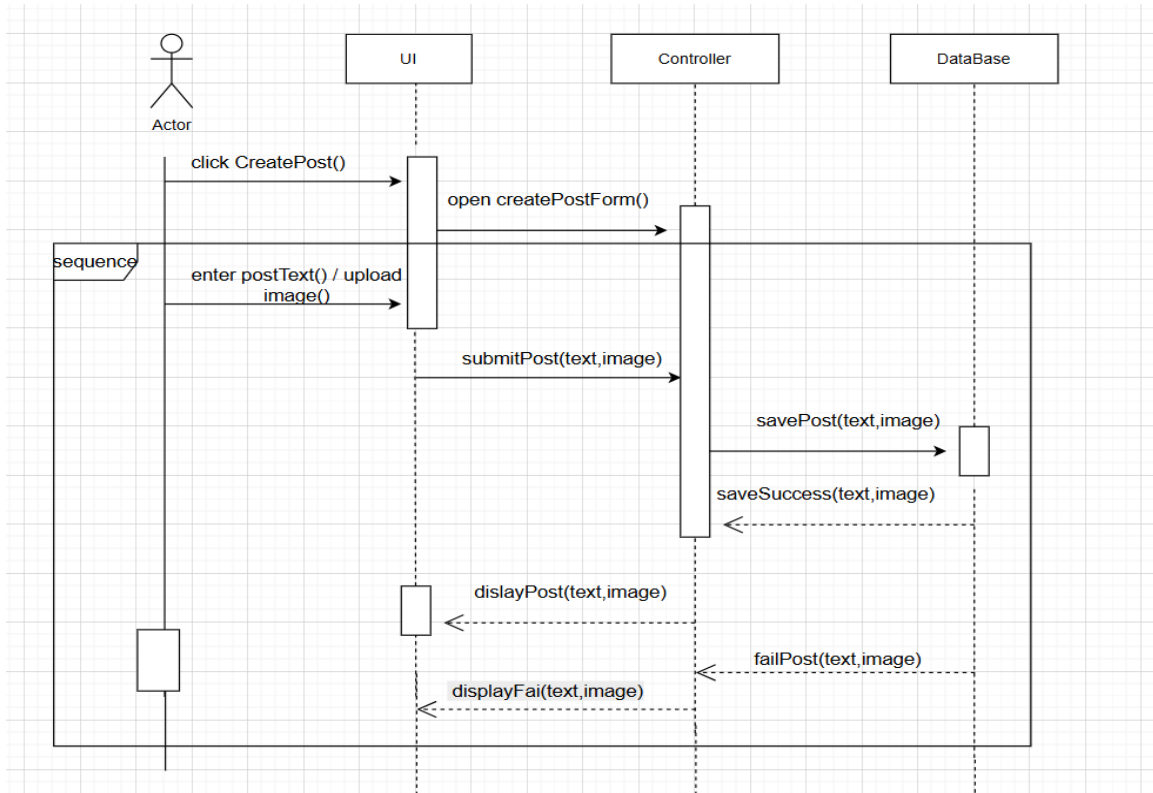
مخطط 2.14 : النشاط حالة البحث عن الأصدقاء



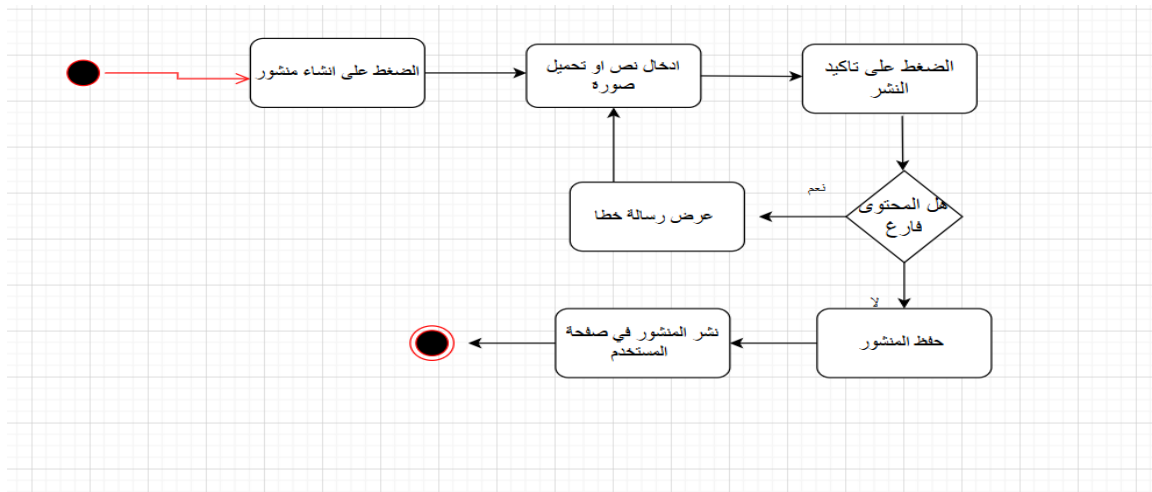
مخطط 2.15 : التسلسل حالة البحث عن الأصدقاء

جدول 2.10 : توصيف حالة نشر منشور جديد

Create Post	UC_09	Use Case Name
المستخدم		Actors
تمكين المستخدم من نشر منشور جديد يحتوي على صورة او نص او كليهما .		Description
1.تسجيل دخول المستخدم إلى حسابه مسبقا.		Precondition
1- يضغط المستخدم على زر "إنشاء منشور جديد" . 2- تظهر نافذة لإدخال النص او تحميل صورة . 3- يكتب المستخدم نص المنشور او يختار صورة . 4- يضغط المستخدم على زر "تأكيد نشر" . 5- يتحقق النظام من صحة البيانات (وجود صورة او نص) . 6- يخزن النظام المنشور في قاعدة البيانات مع بيانات المستخدم والوقت 7- يعرض النظام المنشور الجديد في صفحة المستخدم .		Main flow
المستخدم لم يدخل أي محتوى (لا نص ولا صورة) يظهر تنبيه A4- "المنشور فارغ" A5 - فشل رفع الصورة يظهر تنبيه "حدث خطأ أثناء رفع الصورة"		Alternative flow
تم عرض المنشور في صفحة المستخدم وحفظه في قاعدة البيانات		Postconditions



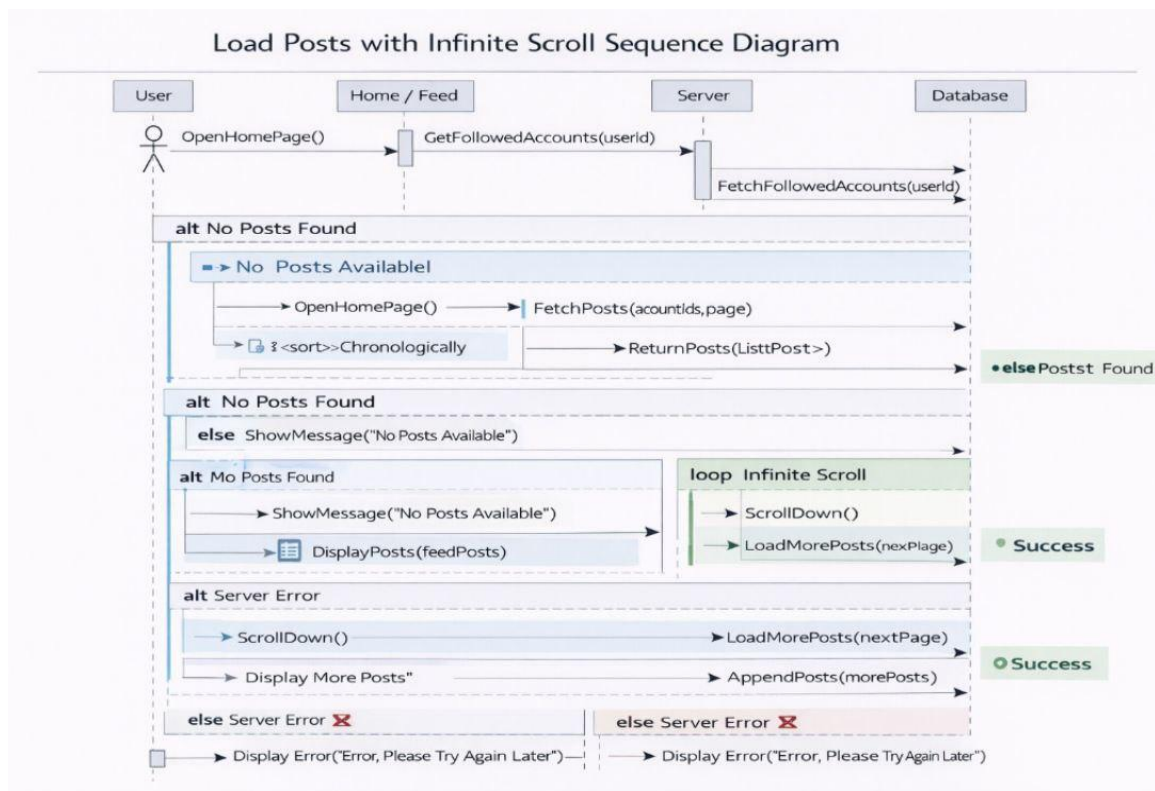
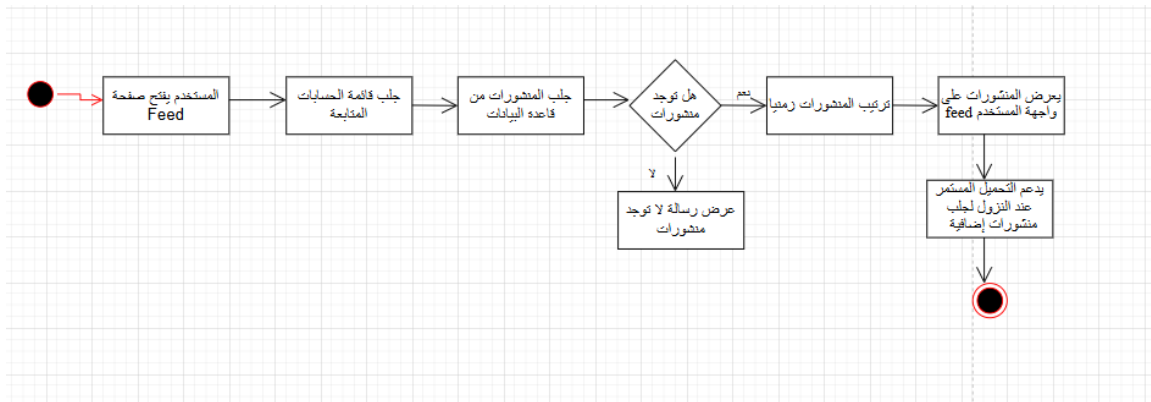
مخطط 2.16 : التسلسل حالة نشر منشور جديد



مخطط 2.17 : النشاط حالة نشر منشور جديد

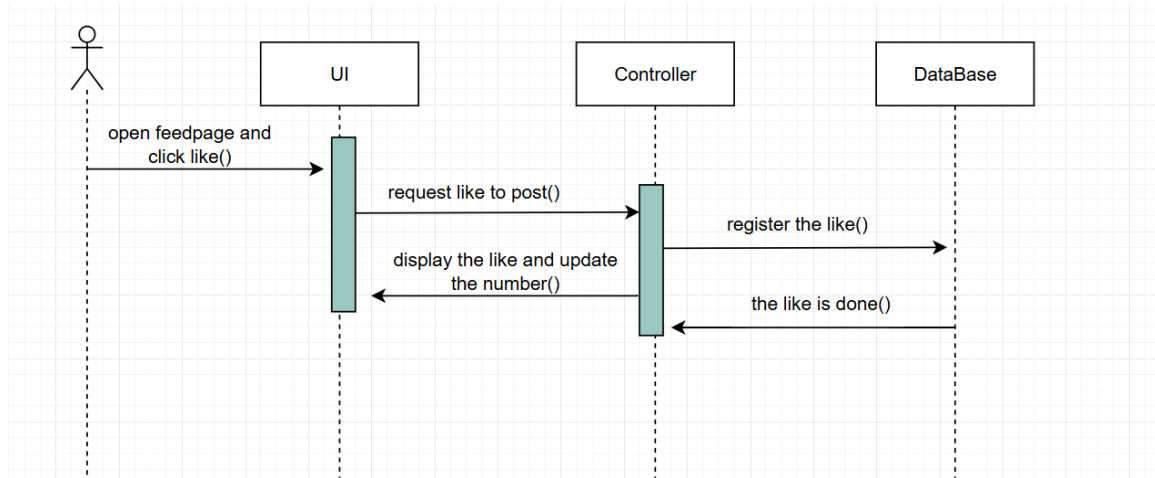
جدول 2.11 : توصيف حالة عرض المنشورات

Load Post	UC_10	Use Case Name
	المستخدم	Actors
يجلب النظام المنشورات من قاعدة البيانات في الصفحة ويعرضها وفق ترتيب زمني مع دعم تحميل المستمر عند النزول.		Description
1.تسجيل دخول المستخدم إلى حسابه مسبقا. 2.المستخدمون لديه متابعون او يتابع حسابات تنشر منشورات.		Precondition
1-يفتح المستخدم الصفحة الرئيسية (Home/Feed). 2-يجلب النظام قائمة الحسابات التي يتابعها المستخدم . 3-يجلب النظام المنشورات من هؤلاء الحسابات من قاعدة البيانات. 4-يرتب المنشورات ترتيبا زمنيا من الاحدث الى الاقدم. 5-يعرض المنشورات على واجهة المستخدم (Feed). 6-يدعم التحميل المستمر عند النزول (infinite scroll) لجلب منشورات إضافية عند الحاجة.		Main flow
لم يجد النظام أي منشورات الحسابات التي يتابعها المستخدم يعرض رسالة: A3: "لا توجد منشورات حاليا". عند نشر منشورات جديدة يقوم النظام بتحديث الصفحة بشكل ديناميكي: A5:		Alternative flow
1-تظهر جميع المنشورات للمستخدم بالترتيب الزمني الصحيح. 2-يتم تحديث الصفحة بشكل مستمر عند وجود منشورات جديدة.		Postconditions

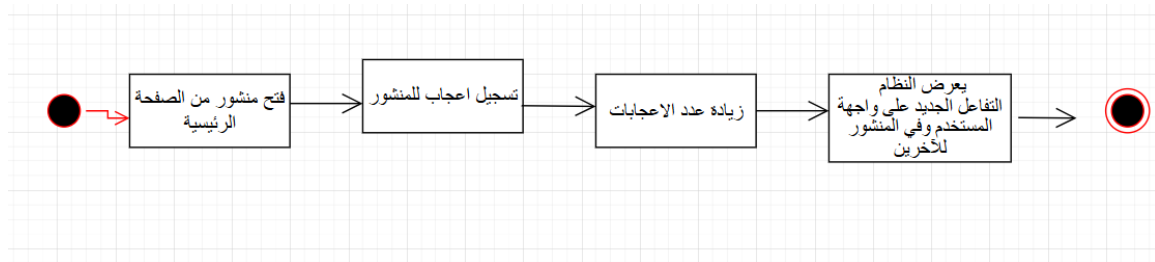


جدول 2.12 : توصيف حالة الإعجاب بالمنشورات

Like Post	UC_11	Use Case Name
	المستخدم	Actors
	يسمح للمستخدم بإظهار رأيه تجاه المنشورات عبر إعطاء إعجاب	Description
	1.تسجيل دخول المستخدم إلى حسابه مسبقا. 2.المنشور موجود ويمكن الوصول إليه للسماح بالتفاعل مع المنشور.	Precondition
	1-يفتح المستخدم المنشورة في الصفحة الرئيسية او الملف الشخصي. 2-يضغط المستخدم زر اعجاب. 3- يسجل النظام التفاعل ويزيد عدد الاعجابات. 4- يعرض النظام التفاعل الجديد على واجهة المستخدم وفي المنشور للآخرين.	Main flow
	يعرض النظام رسالة :هذا المنشور غير متاح:A3	Alternative flow
	يزداد عدد الاعجابات على المنشور ويظهر للمستخدمين الآخرين .	Postconditions



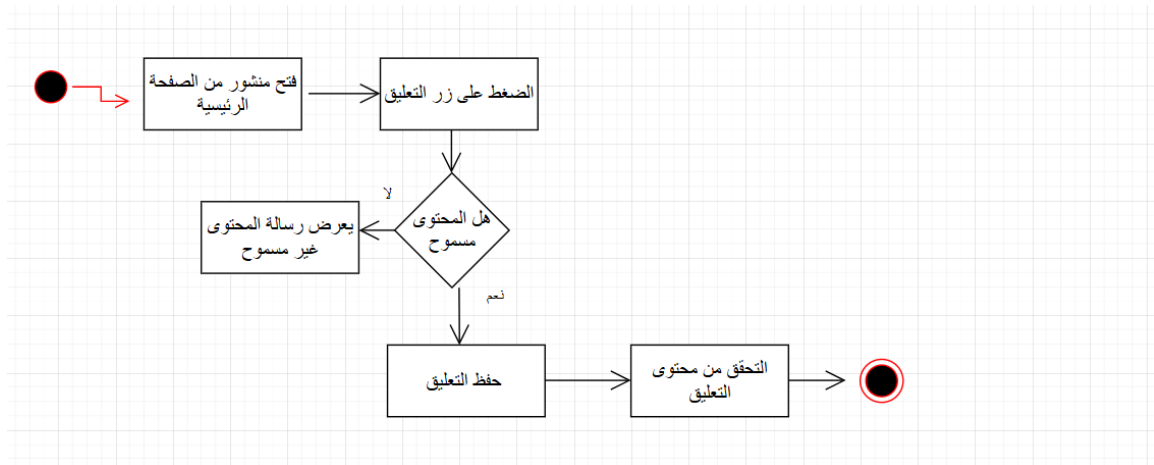
مخطط 2.20 : التسلسل حالة الإعجاب بالمنشورات



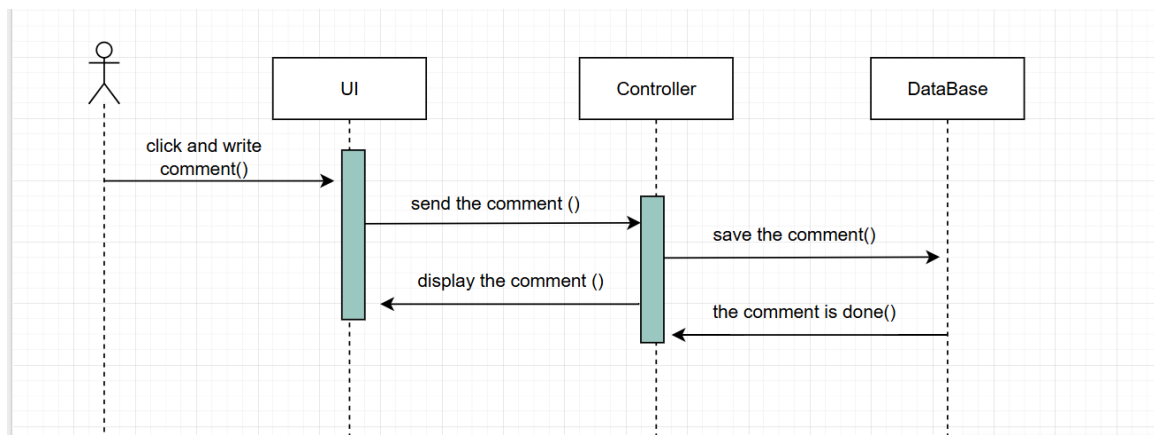
مخطط 2.21 : النشاط حالة الإعجاب بالمنشورات

جدول 2.13 : توصيف حالة التعليق على المنشورات

Comment Post	UC_12	Use Case Name
المستخدم		Actors
يسمح للمستخدم بإظهار رأيه تجاه المنشورات عبر إضافة تعليق .		Description
1.تسجيل دخول المستخدم إلى حسابه مسبقا. 2.المنشور موجود ويمكن الوصول إليه للسماح بالتفاعل مع المنشور.		Precondition
1-يفتح المستخدم المنشورة في الصفحة الرئيسية او الملف الشخصي. 2-يضغط المستخدم زر تعليق . 3-يتحقق النظام من وجود المنشور وصلاحيه الوصول اليه. 4-يتحقق النظام من محتوى التعليق ثم يحفظه في قاعدة البيانات. 5-يعرض النظام التفاعل الجديد على واجهة المستخدم وفي المنشور للآخرين.		Main flow
A3: يعرض النظام رسالة :هذا المنشور غير متاح. النظام يرفض التعليقات ويعرض رسالة "التعليق يحتوي على محتوى غير A5: مسموح به".		Alternative flow
يتم حفظ التعليق في قاعدة البيانات ويظهر علو المنشور بالترتيب الزمني الصحيح.		Postconditions



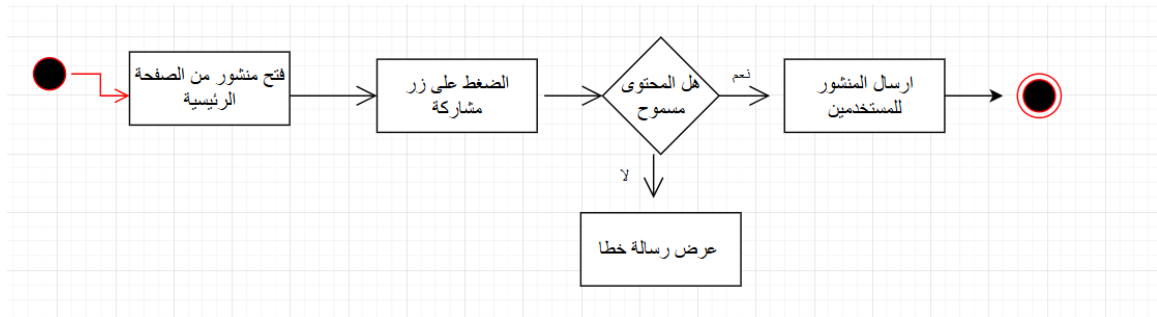
مخطط 2.22 : النشاط حالة التعليق على المنشورات



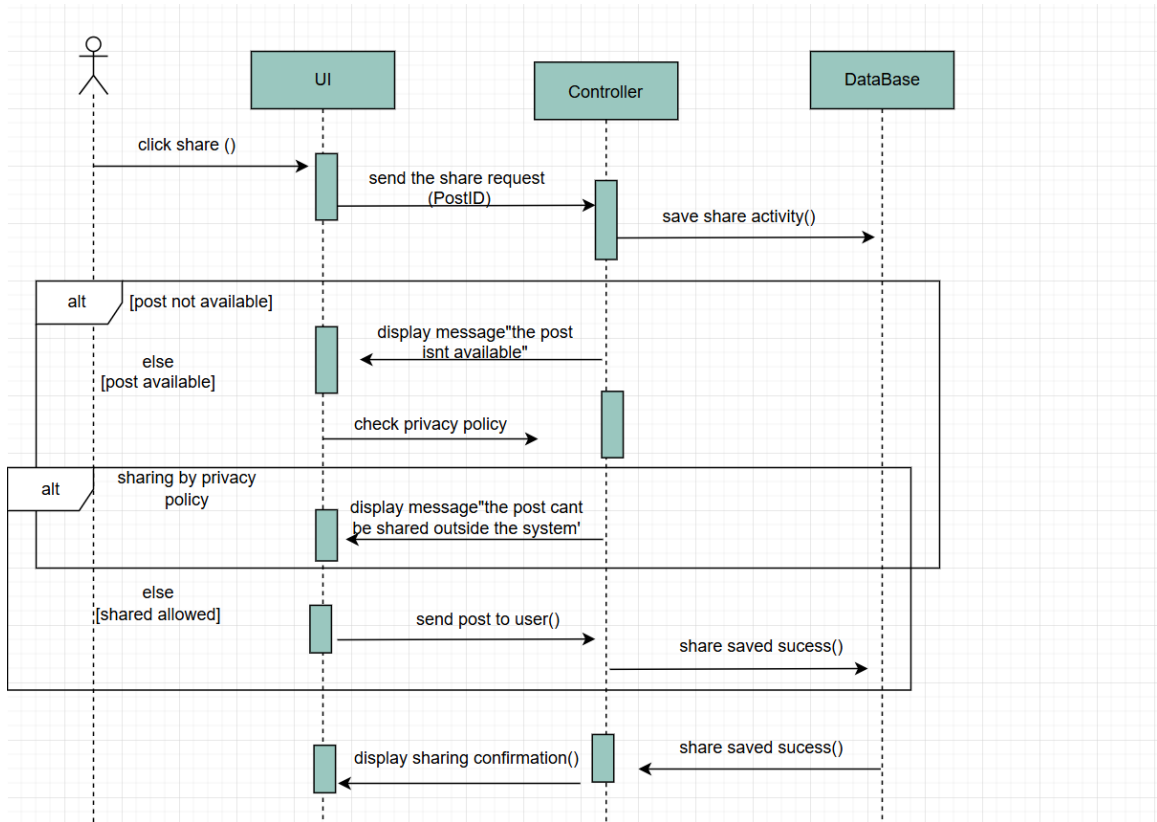
مخطط 2.23 : التسلسل حالة التعليق على المنشورات

جدول 2.14 : توصيف حالة مشاركة المنشورات

Share Post	UC_13	Use Case Name
المستخدم		Actors
يتيح للمستخدم لتحكم بالمنشورات التي يود مشاركتها.		Description
1.تسجيل دخول المستخدم إلى حسابه مسبقا.		Precondition
1-يفتح المستخدم المنشورة في الصفحة الرئيسية او الملف الشخصي.. 2-يضغط المستخدم زر للمشاركة . 3-يتحقق النظام من وجود المنشور وصلاحيه الوصول اليه. 4-يقوم المستخدم بنشر المنشور للمستخدمين .		Main flow
يعرض النظام رسالة :هذا المنشور غير متاح:A3 إذا كانت سياسة الخصوصية تمنع مشاركة المنشور خارجيًا، يظهر النظام :A5 رسالة: "لا يمكن مشاركة هذا المنشور خارج النظام".		Alternative flow
يتم انشاء رابط للمشاركة او ارسال المنشور للمستخدمين الاخرين وتسجل عملية المشاركة في النظام.		Postconditions



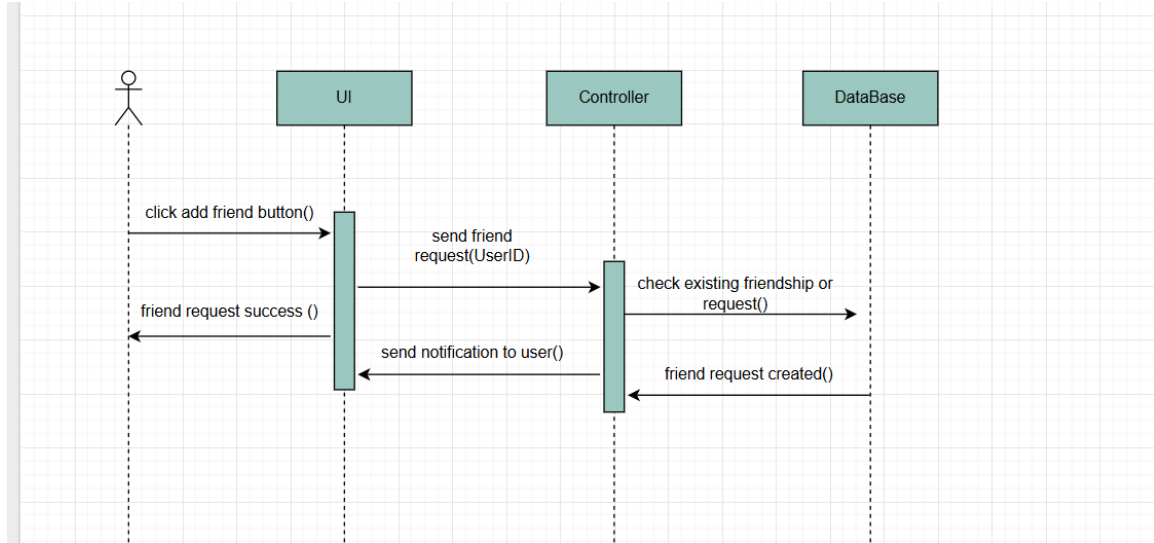
مخطط 2.24 : النشاط حالة مشاركة المنشورات



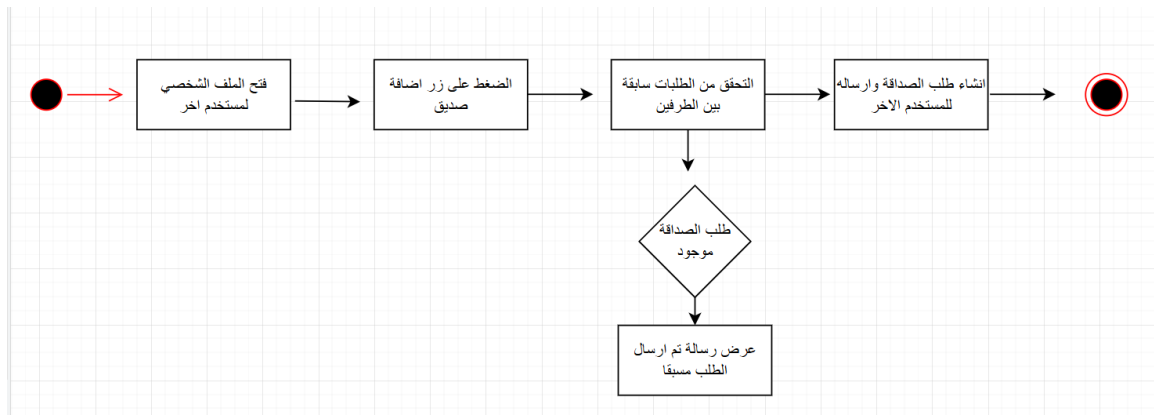
مخطط 2.25 : التسلسل حالة مشاركة المنشورات

جدول 2.15 : توصيف حالة إضافة صديق

Friend Request	UC_14	Use Case Name
المستخدم		Actors
ارسال طلب صداقة الى المستخدم .		Description
1.تسجيل دخول المستخدم إلى حسابه مسبقا.		Precondition
1-يفتح المستخدم الملف الشخصي لمستخدم اخر. 2-يضغط المستخدم على زر إضافة صديق. 3-النظام يتحقق من عدم وجود صداقة او طلب سابق بين الطرفين. 4-النظام ينشئ طلب صداقة جديد . 5-النظام يرسل اشعارا للمستخدم الاخر بوجود طلب صداقة. 6- النظام يعرض رسالة بان طلب الصداقة قد تم ارساله.		Main flow
طلب الصداقة موجود مسبقا النظام يعرض رسالة تم ارسال طلب مسبقا: A3		Alternative flow
يتم انشاء طلب صداقة.		Postconditions



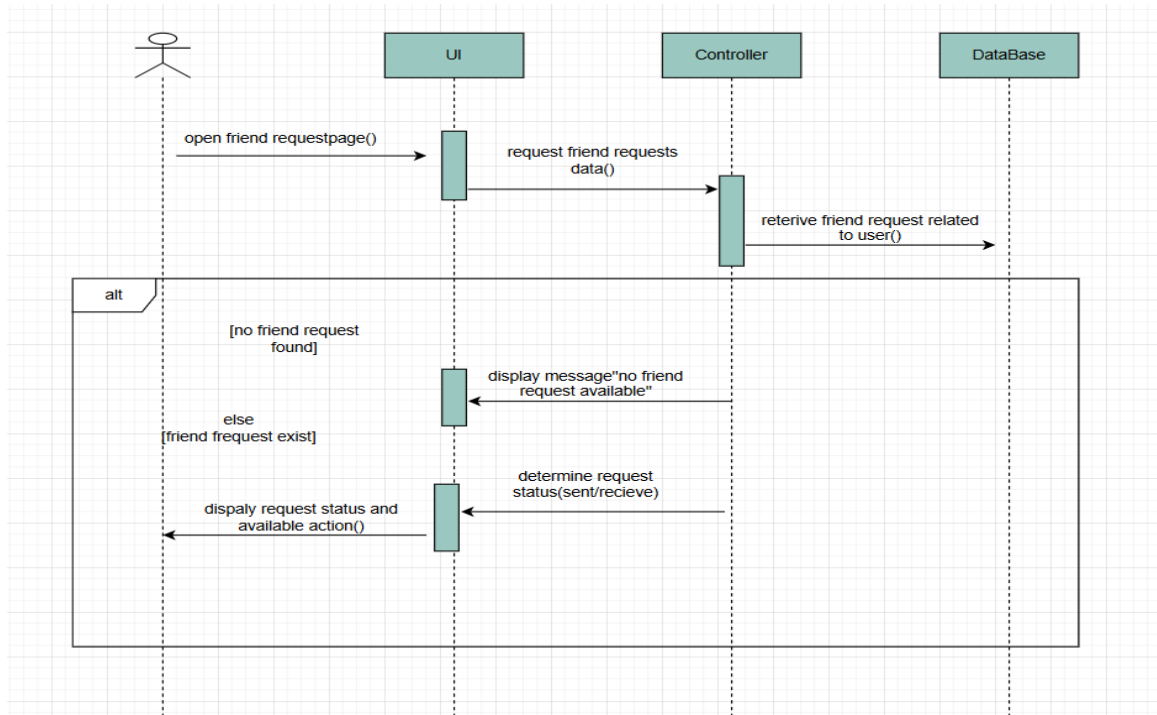
مخطط 2.26 : التسلسل حالة إضافة صديق



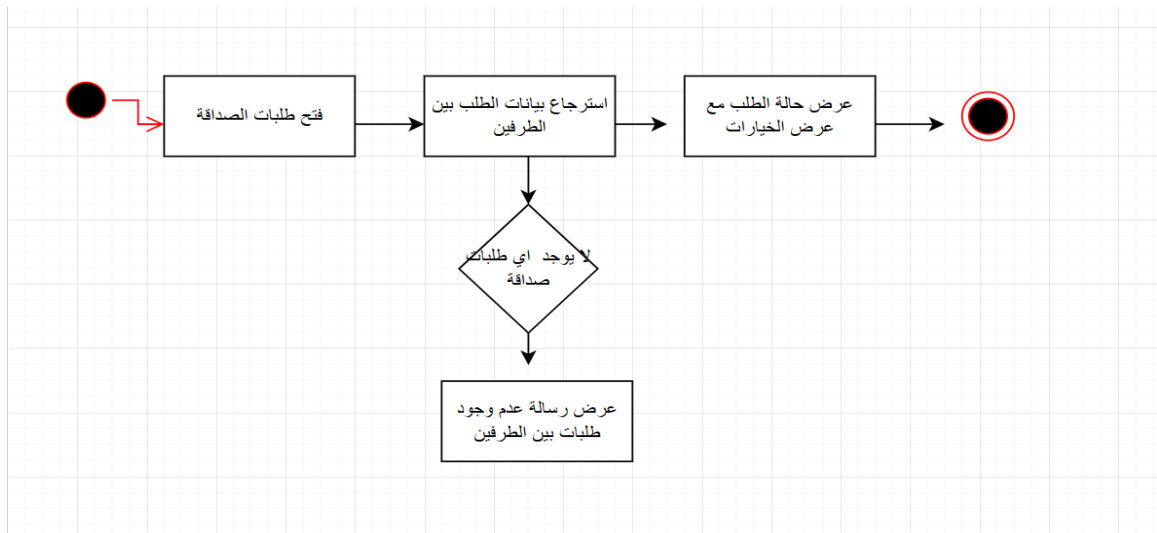
مخطط 2.27 : النشاط حالة إضافة صديق

جدول 2.16 : توصيف معرفة حالة طلب الصداقة

Order Status	UC_15	Use Case Name
المستخدم		Actors
يتيح للمستخدم معرفة حالة طلب الصداقة الذي قام بارساله او استقبله .		Description
1.تسجيل دخول المستخدم إلى حسابه مسبقا.		Precondition
1. المستخدم يفتح صفحة طلبات الصداقة . 2.النظام يسترجع بيانات طلب الصداقة المرتبط بين المستخدمين. 3. النظام يتحقق ويعرض حالة الطلب مع عرض الخيارات المتاحة(الغاء او قبول او رفض).		Main flow
لا يوجد أي طلبات صداقة: A2		Alternative flow
عرض حالة طلب المستخدم.		Postconditions



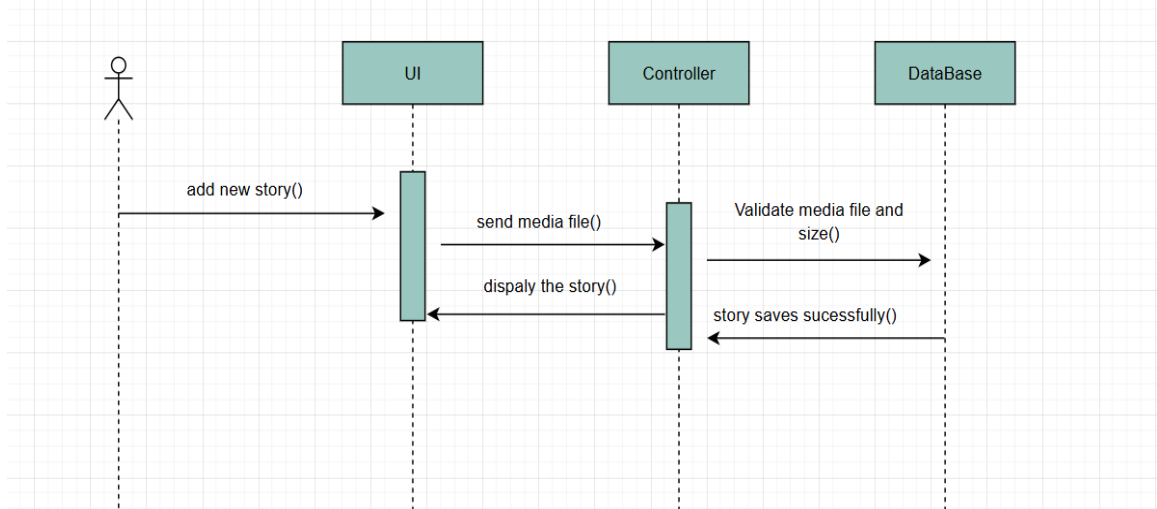
مخطط 2.28 : التسلسل معرفة حالة طلب الصداقة



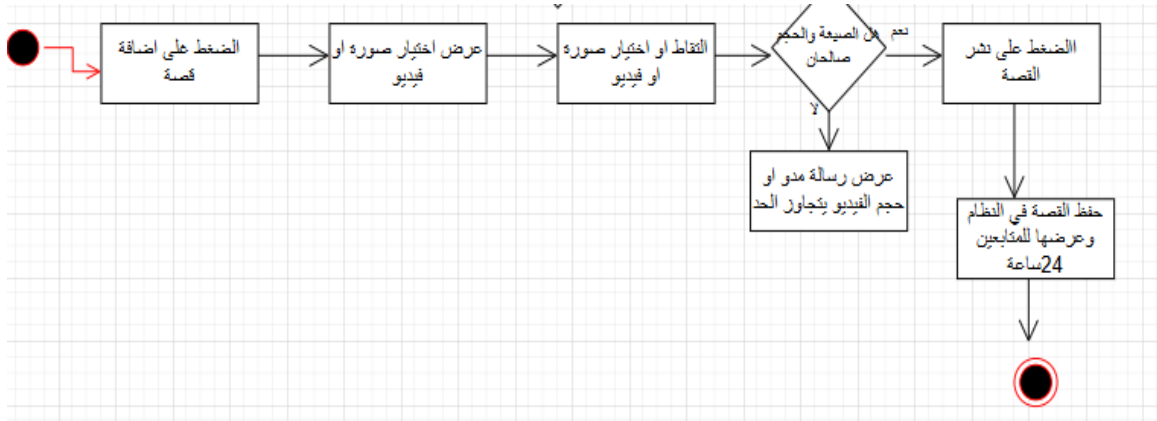
مخطط 2.29 : النشاط معرفة حالة طلب الصداقة

جدول 2.17 : توصيف حالة إضافة قصة جديدة

Create Story	UC_16	Use Case Name
المستخدم		Actors
يتيح للمستخدم رفع قصة جديدة على حسابه بصيغة صورة أو فيديو قصير، بحيث تكون متاحة للمشاهدة لفترة زمنية محددة (24 ساعة)، مما يعزز التفاعل والمشاركة داخل المنصة.		Description
1. تسجيل دخول المستخدم إلى حسابه مسبقاً.		Precondition
1. المستخدم يضغط على إضافة قصة جديدة. 2. النظام يعرض خيار اختيار صورة أو فيديو قصير. 3. المستخدم يلتقط صورة/فيديو أو يختار من المعرض. 4. النظام يتحقق من صيغة وحجم الملف. 5. المستخدم يضيف (نص، ملصقات، فلاتر – اختياري) ويضغط على نشر القصة. 6. النظام ينشر القصة ويجعلها مرئية لمدة 24 ساعة.		Main flow
حجم الفيديو أكبر من المسموح يعرض رسالة: "مدة أو حجم الفيديو A4 يتجاوز الحد المسموح".		Alternative flow
يتم نشر القصة بنجاح وتصبح مرئية للمتابعين أو الأصدقاء حسب إعدادات الخصوصية.		Postconditions



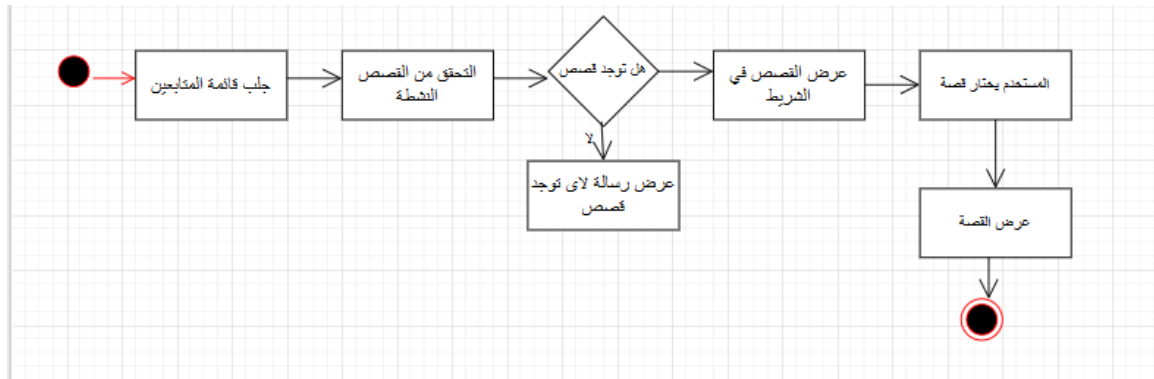
مخطط 2.30 : التسلسل حالة إضافة قصة جديدة



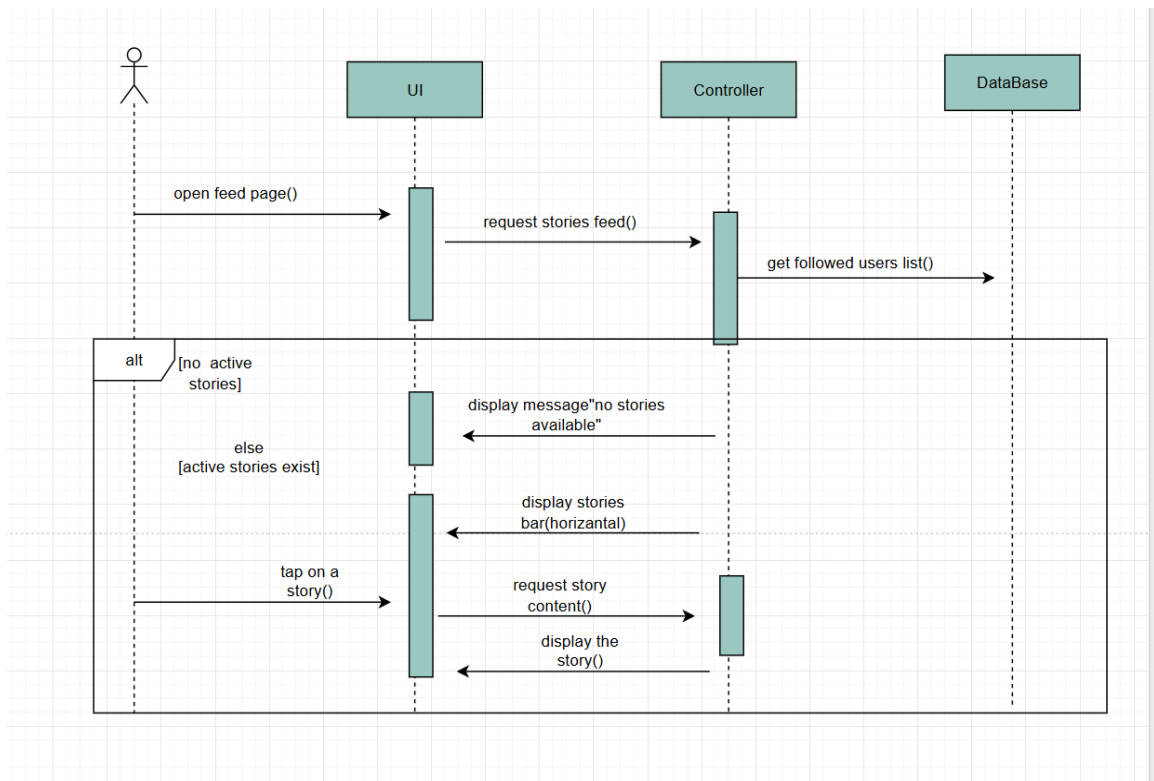
مخطط 2.31 : النشاط حالة إضافة قصة جديدة

جدول 2.18 : توصيف عرض القصص المتاحة للمتابعين

Display User Story	UC_17	Use Case Name
المستخدم		Actors
يتيح للمستخدم عرض القصص المنشورة من قبل المستخدمين الذين يتابعهم فقط، وذلك وفق إعدادات الخصوصية ومدة صلاحية القصة، مما يضمن تجربة مشاهدة مخصصة وأمنة.		Description
1. تسجيل دخول المستخدم إلى حسابه مسبقاً.		Precondition
1. المستخدم يدخل إلى الصفحة الرئيسية. 2. النظام يجلب قائمة المستخدمين الذين يتابعهم المستخدم و يتحقق من وجود قصص نشطة. 3. النظام يعرض القصص في شريط علوي بشكل أفقي. 4. المستخدم يضغط على قصة أحد المتابعين. 5. النظام يعرض القصة (صورة أو فيديو). 6. بعد انتهاء القصة، النظام ينتقل تلقائياً إلى القصة التالية.		Main flow
لا توجد قصص نشطة: A2: القصة محجوبة بالخصوصية القصة غير مسموح بعرضها لهذا المستخدم: A2: انتهاء مدة القصة أثناء المشاهدة وتنتهي مدة صلاحية القصة: A6:		Alternative flow
يتم عرض القصص المتاحة للمتابعين بترتيب زمني.		Postconditions



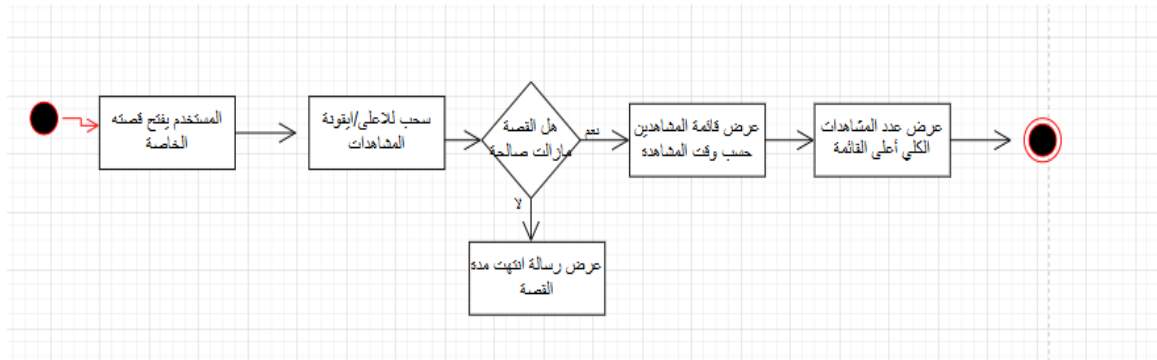
مخطط 2.32 : النشاط حالة عرض القصص المتاحة للمتابعين



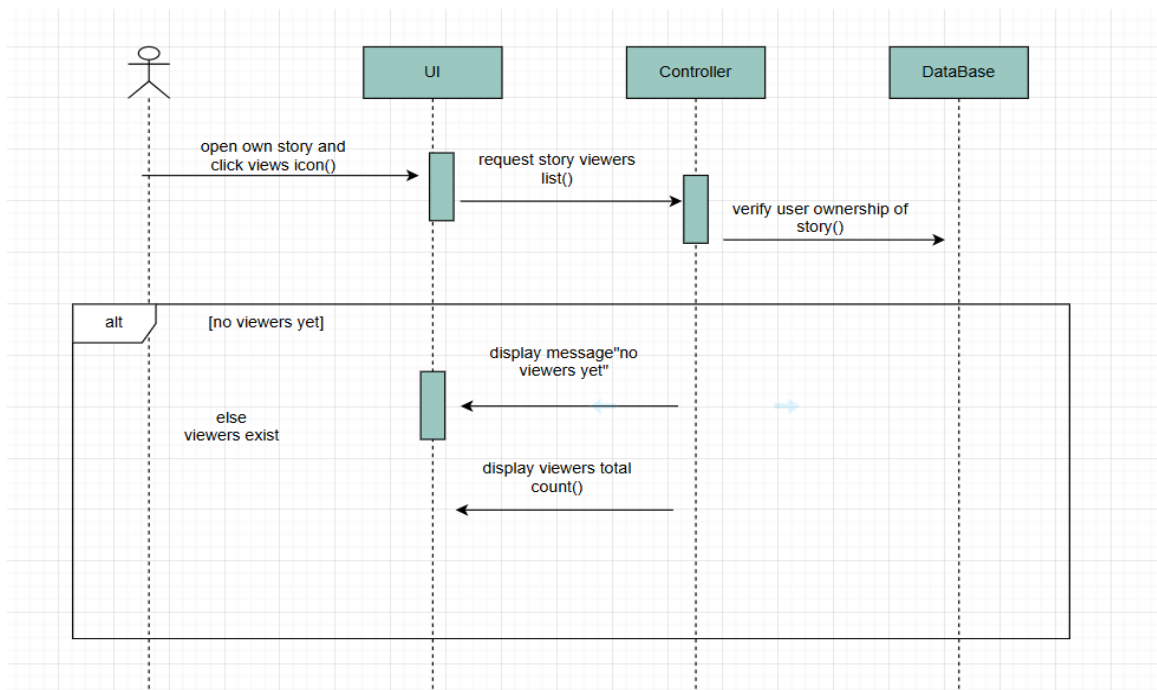
مخطط 2.33 : التسلسل حالة عرض القصص المتاحة للمتابعين

جدول 2.19 : توصيف عرض قائمة المشاهدين

View User List	UC_18	Use Case Name
المستخدم		Actors
يُتيح لصاحب القصة عرض قائمة المستخدمين الذين قاموا بمشاهدة قصته، مما يوفر شفافية حول التفاعل مع المحتوى المؤقت داخل المنصة.		Description
1. تسجيل دخول المستخدم إلى حسابه مسبقاً.		Precondition
1. المستخدم يفتح قصته الخاصة. 2. المستخدم يسحب للأعلى أو يضغط على أيقونة المشاهدين. 3. النظام يتحقق من صلاحية المستخدم لعرض القائمة. 4. النظام يعرض قائمة المشاهدين مرتبة حسب وقت المشاهدة. 5. النظام يعرض عدد المشاهدات الكلي أعلى القائمة.		Main flow
A1: انتهاء مدة القصة (24 ساعة). A2: لا يوجد مشاهدون لا توجد أي مشاهدة بعد.		Alternative flow
يتم عرض قائمة المشاهدين بشكل صحيح مع وقت المشاهدة		Postconditions



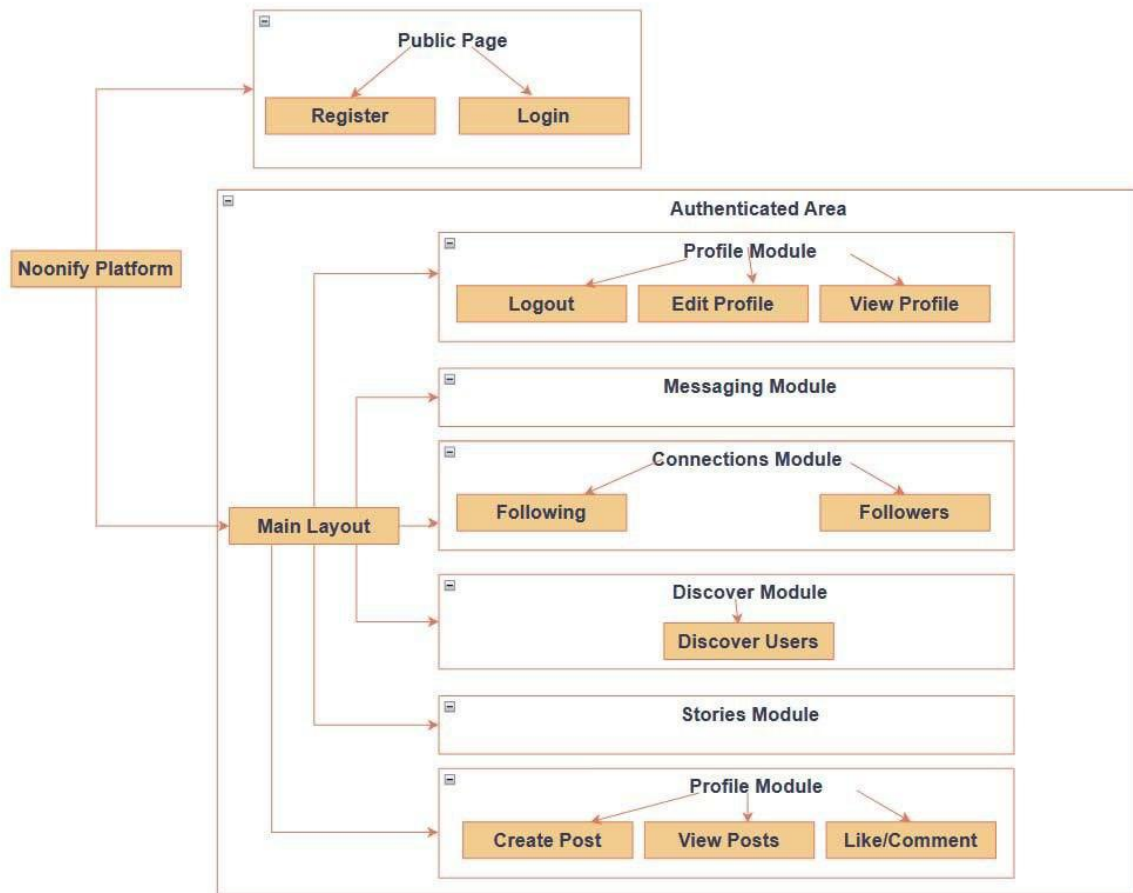
مخطط 2.34 : النشاط عرض قائمة المشاهدين



مخطط 2.35 : النشاط عرض قائمة المشاهدين

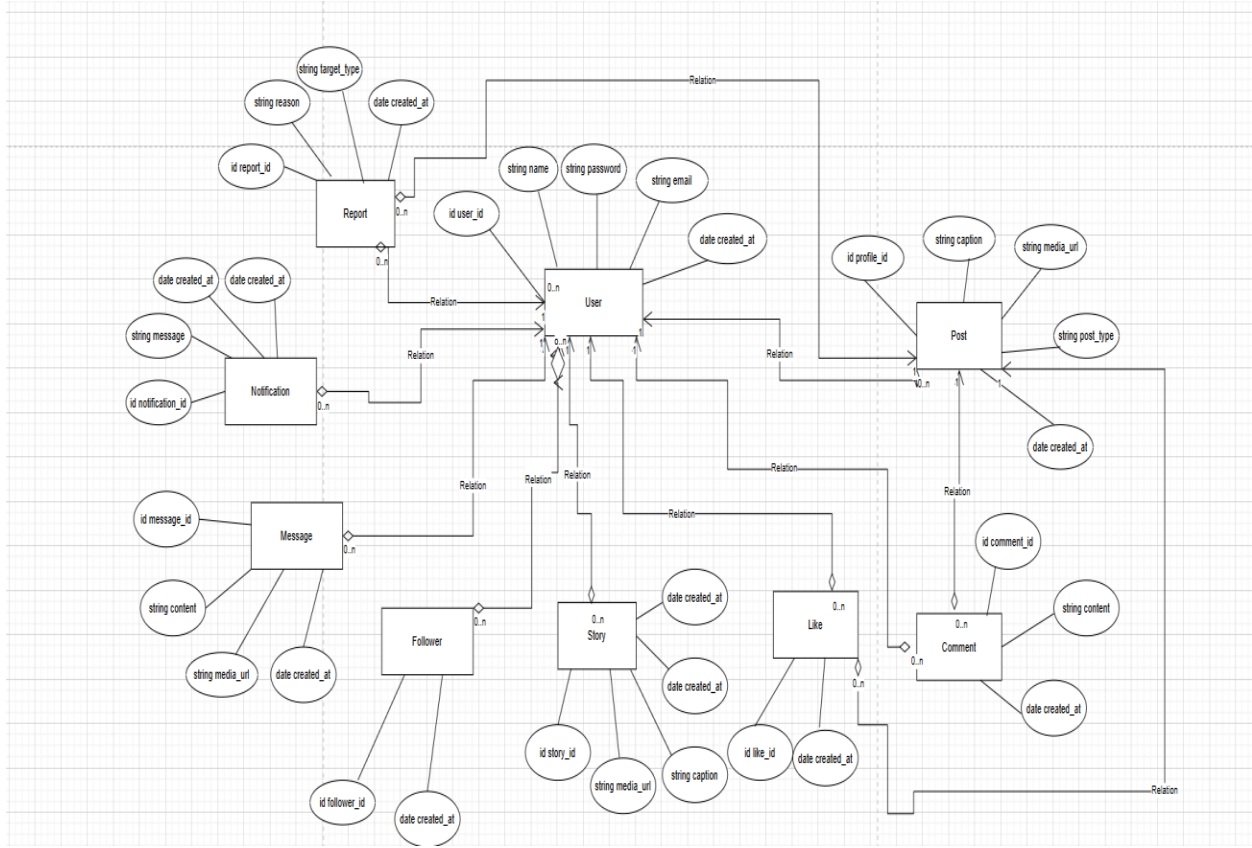
3 الفصل الثالث : الدراسة التصميمية

3.1 مخطط ال Site Map



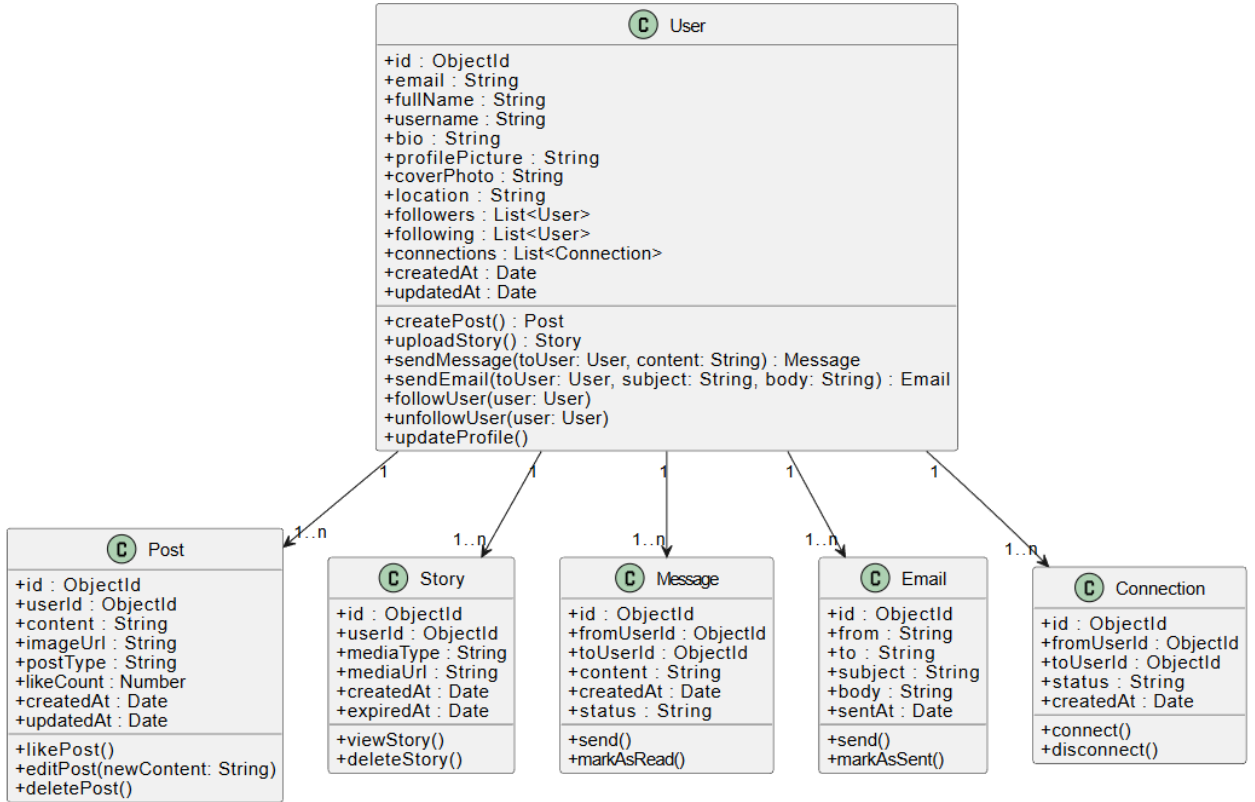
الشكل 3.6 : ال Site Map

3.2 مخطط قاعدة البيانات ERD



الشكل 3.7 : ال ERD

3.3 مخطط الصفوف Class Diagram



الشكل 3.8 : ال Class Diagram

3.4 جدول Test Cases

جدول 3.20: Test Cases

Test Case ID	Test Case Description	Preconditions	Test Steps	Expected Result
TC-01	User registration using email or phone	User not registered	Open app → Register → Enter email/phone → Submit	Account created successfully
TC-02	Login using Google/GitHub	User has valid OAuth account	Open app → Login → Select Google/GitHub	User logged in successfully
TC-03	Reset forgotten password	User registered	Click Forgot Password → Enter email	Password reset link sent
TC-04	Logout from current session	User logged in	Click Logout	User session terminated
TC-05	View user profile information	User logged in	Open Profile Page	Profile details displayed
TC-06	Edit and update profile	User logged in	Edit profile → Save changes	Profile updated successfully
TC-07	View other users' profiles	User logged in	Search user → Open profile	Other user profile displayed
TC-08	Display online/offline status	Multiple users exist	View users list	Correct status shown
TC-09	Create new post	User logged in	Create post → Publish	Post published
TC-10	Edit or delete post	Post exists	Edit/Delete selected post	Post updated or deleted
TC-11	View feed sorted chronologically	Multiple posts exist	Open feed	Posts ordered by time
TC-12	Like a post	Post exists	Click Like	Like registered
TC-13	Save or share post	Post exists	Click Save/Share	Post saved/shared
TC-14	Report inappropriate post	Post violates policy	Click Report	Report submitted

TC-15	Upload story (image/video)	User logged in	Upload story	Story uploaded
TC-16	View stories from followed users	Stories available	Open stories section	Stories displayed
TC-17	Auto delete story after 24 hours	Story older than 24h	System auto process	Story removed
TC-18	View story viewers list	Story has viewers	Open viewers list	Viewers displayed

(Requirements Traceability Matrix) RTM جدول 3.5

جدول 3.21 : ال RTM

Req ID	Title and Dependency	SRS Section	Coding Module	System Test Case
FR-01	Register New Account – depends on Auth module	3.1	auth_service, user_model	TC-01
FR-02	Login using Email/Google/GitHub – depends on Auth module	3.2	auth_service, oauth_service	TC-02
FR-03	Reset Password – depends on Auth module	3.3	password_service	TC-03
FR-04	Logout – depends on Session manager	3.4	session_manager	TC-04
FR-05	View User Profile – depends on Profile module	3.5	profile_service	TC-05
FR-06	Edit User Profile – depends on Profile module	3.6	profile_service	TC-06
FR-07	View Other Users Profiles – depends on Profile module	3.7	profile_service	TC-07

FR-08	Show Online Status – depends on Presence module	3.8	presence_service	TC-08
FR-09	Create Post – depends on Post module	3.9	post_service	TC-09
FR-10	Edit/Delete Post – depends on Post module	3.10	post_service	TC-10
FR-11	View Feed Chronologically – depends on Feed module	3.11	feed_service	TC-11
FR-12	Like Post – depends on Interaction module	3.12	interaction_service	TC-12
FR-13	Save/Share Post – depends on Interaction module	3.13	interaction_service	TC-13
FR-14	Report Post – depends on Moderation module	3.14	moderation_service	TC-14
FR-15	Upload Story – depends on Story module	3.15	story_service	TC-15
FR-16	View Stories – depends on Story module	3.16	story_service	TC-16
FR-17	Delete Story after 24h – depends on Story module	3.17	story_service, scheduler	TC-17
FR-18	View Story Viewers – depends on Story module	3.18	story_service	TC-18

4. الفصل الرابع : تنفيذ المشروع

4.1 الواجهات

My account | My Application

light-walrus-75.accounts.dev/sign-up

Gmail YouTube Maps News Translate

Create your account
Welcome! Please fill in the details to get started.

Continue with Google

or

First name Optional Last name Optional

Sara Eyad

Email address

saraob771@gmail.com

Password

✓ Your password meets all the necessary requirements.

Continue

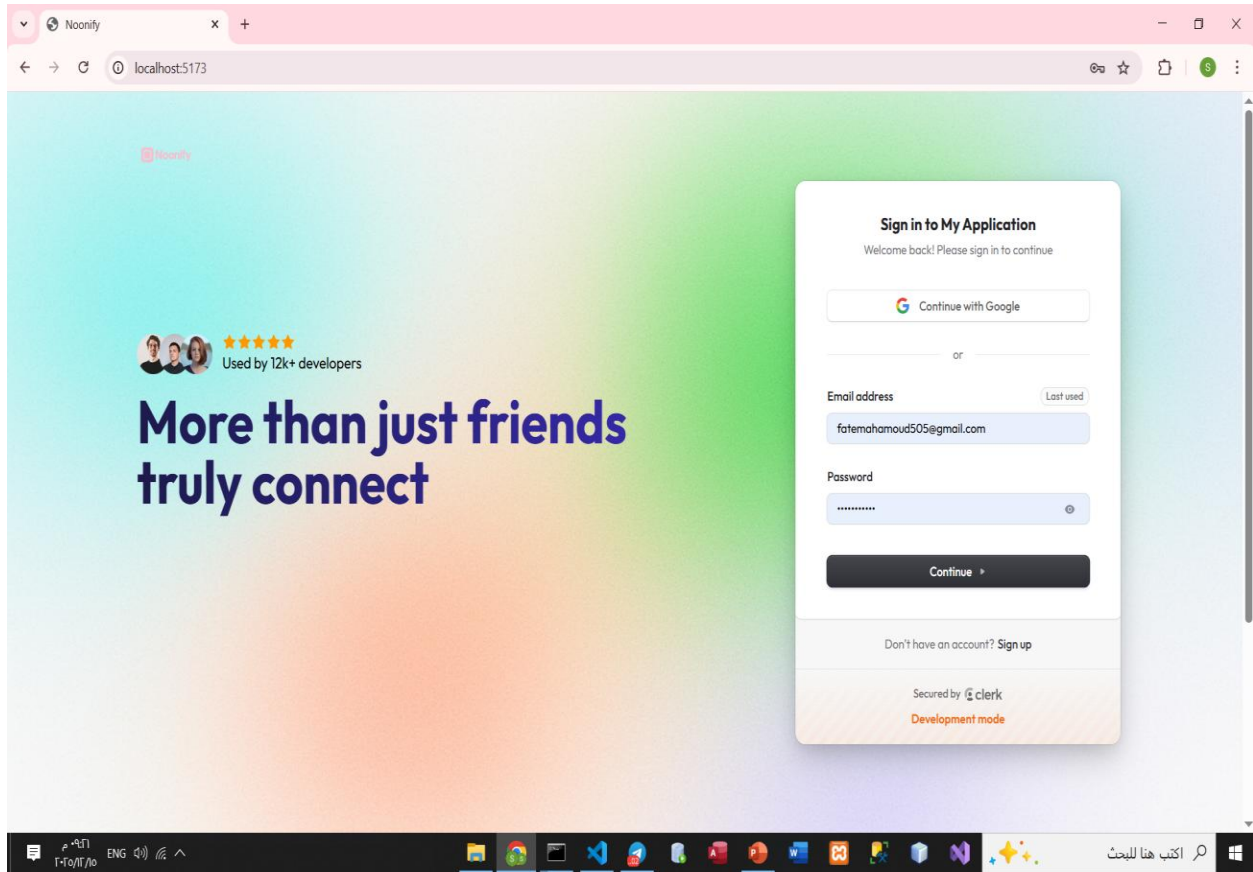
Already have an account? [Sign in](#)

Sponsored by Clerk

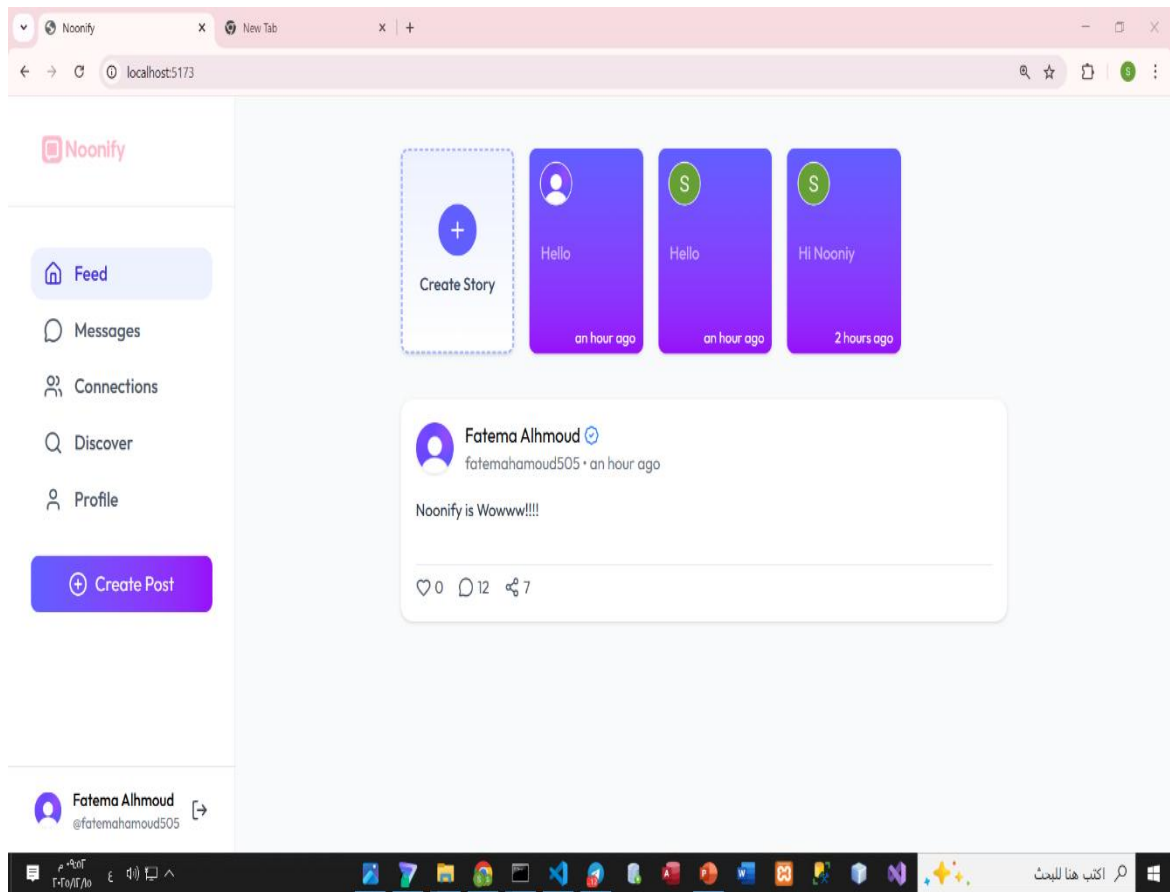
9:51 ENG 10/10/20

اكتب هنا للبحث

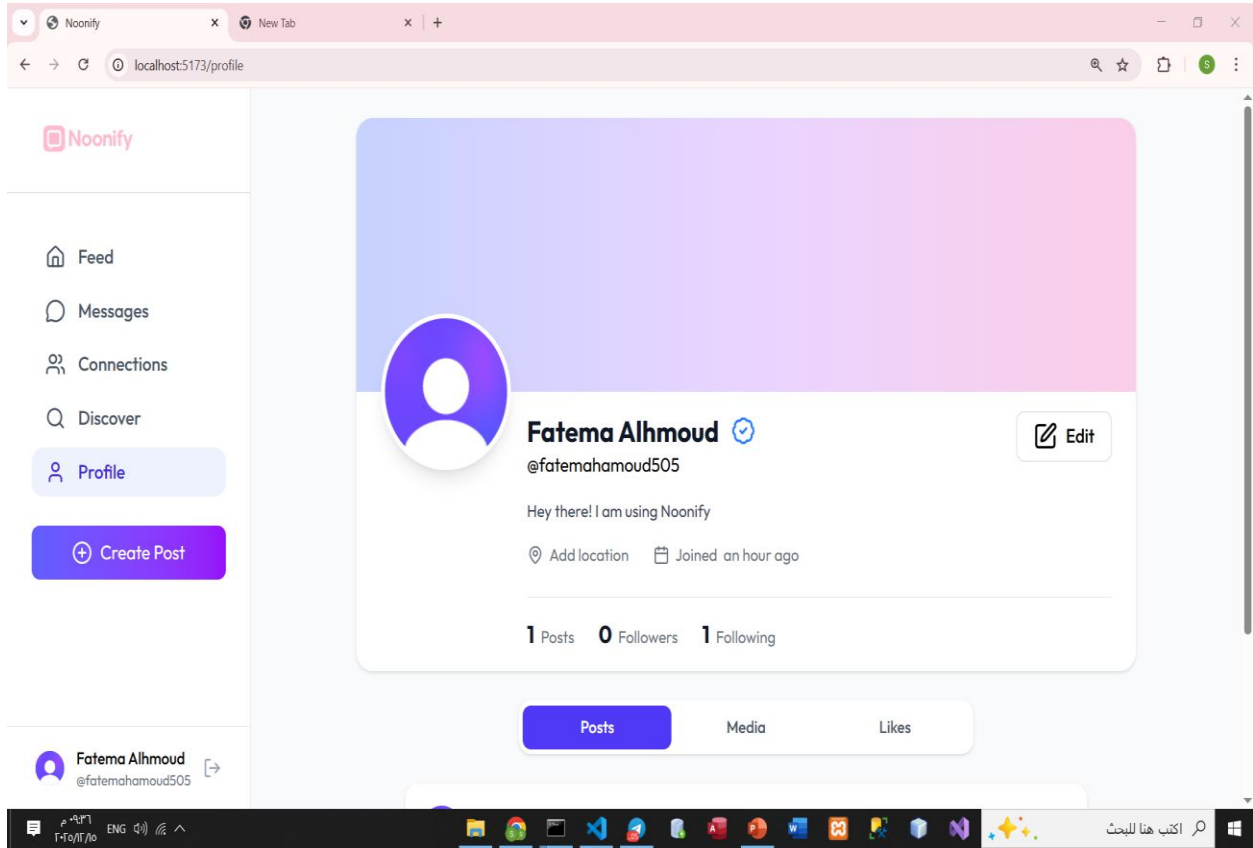
الشكل 4.9 : (1.1) واجهة تسجيل حساب جديد



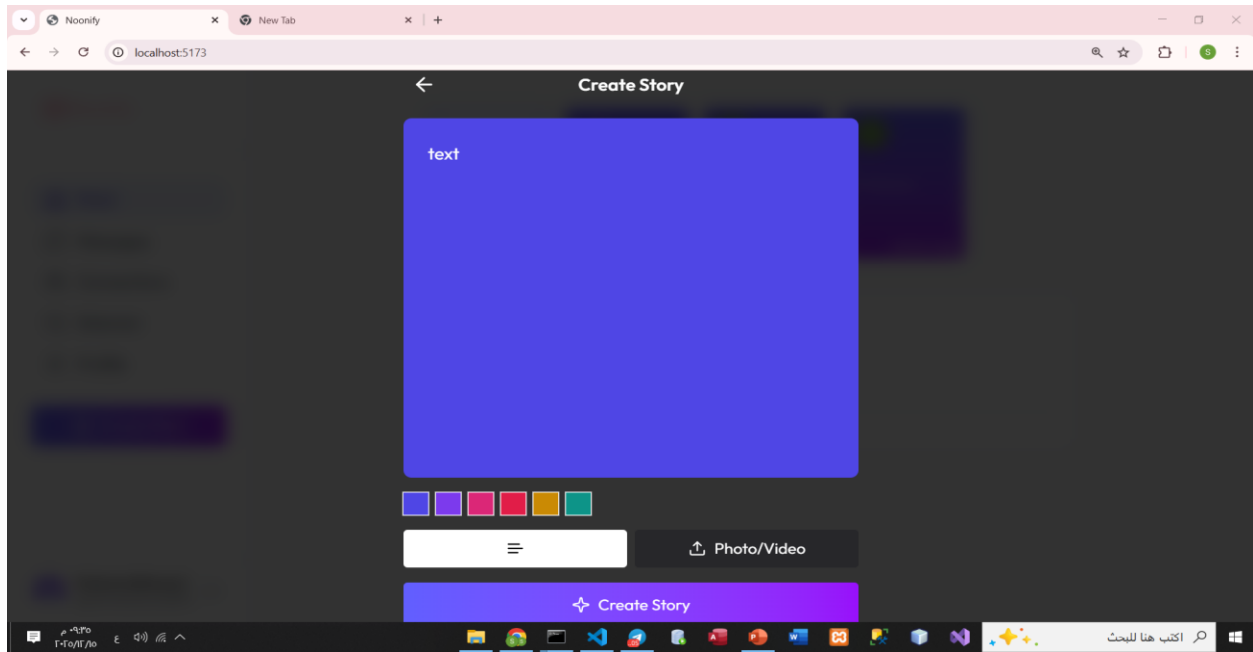
الشكل 4.10 : واجهة تسجيل الدخول (1.2)



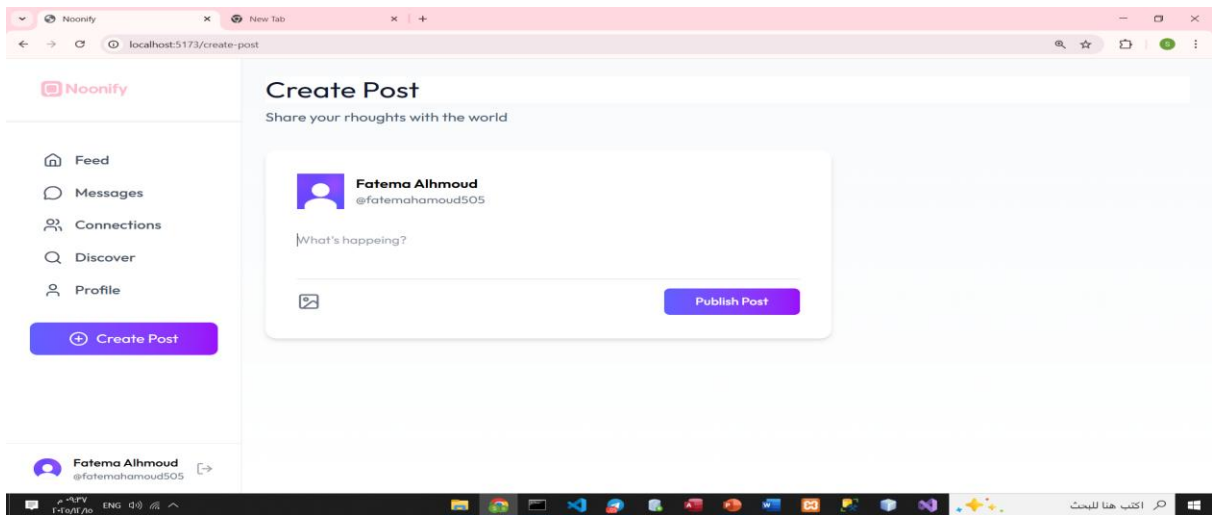
الشكل 4.11 : (1.3) زر تسجيل الخروج



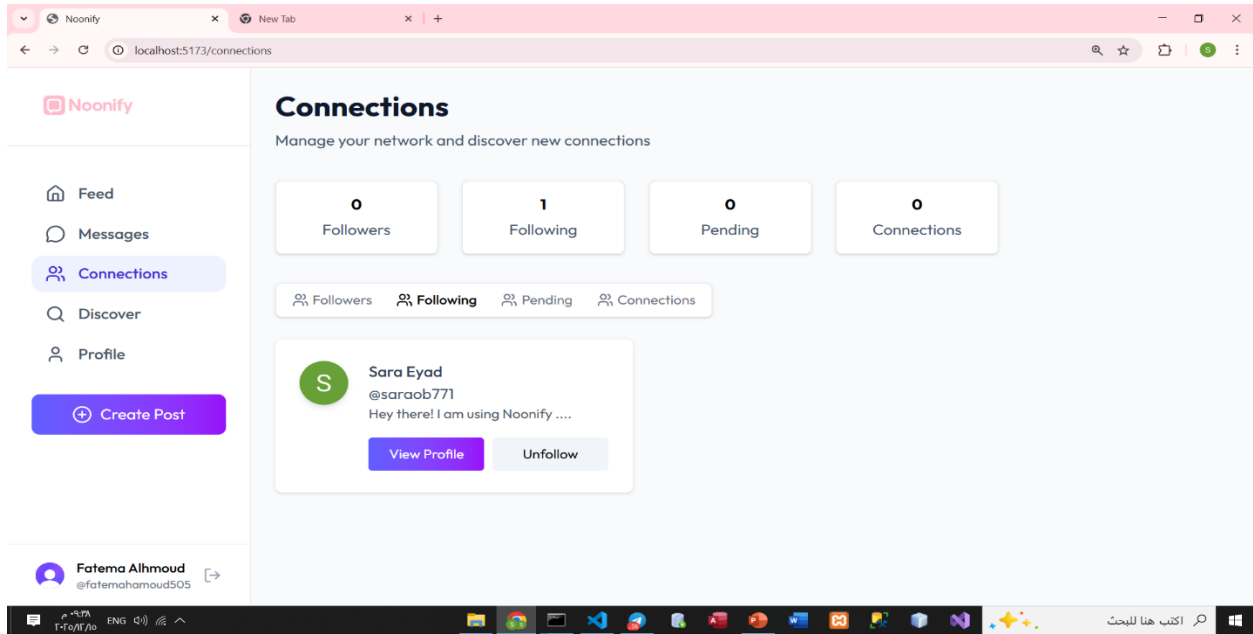
الشكل 4.12 : (1.4) عرض المعلومات الأساسية للمستخدم



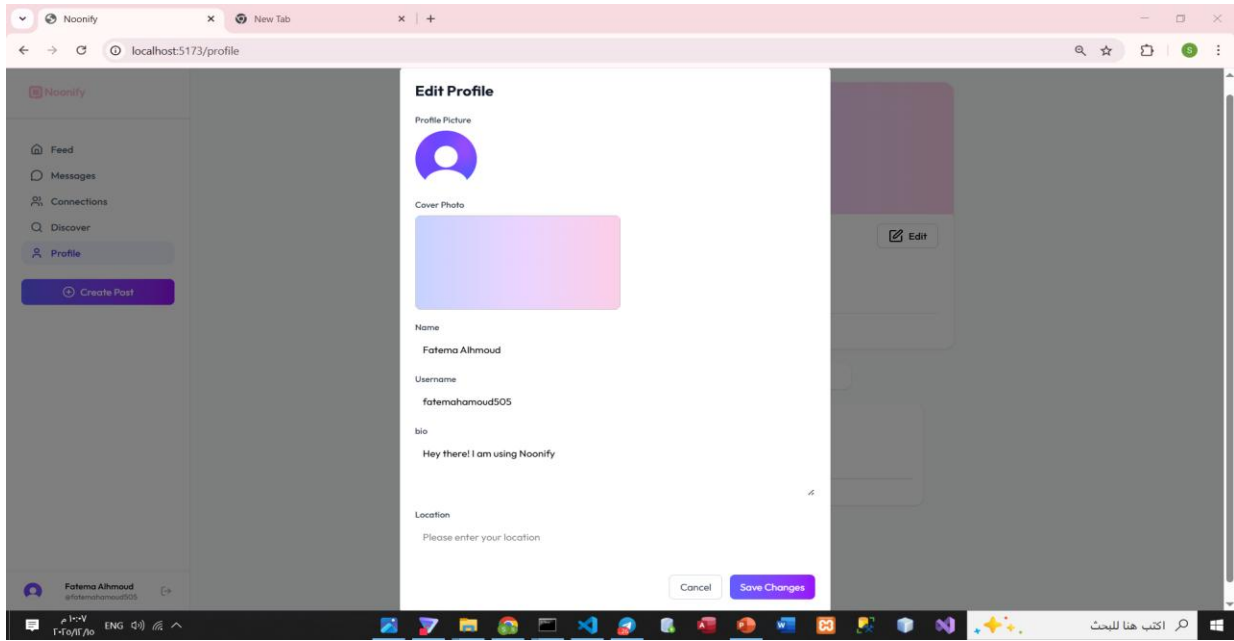
الشكل 4.13 : (1.5) رفع قصة جديدة



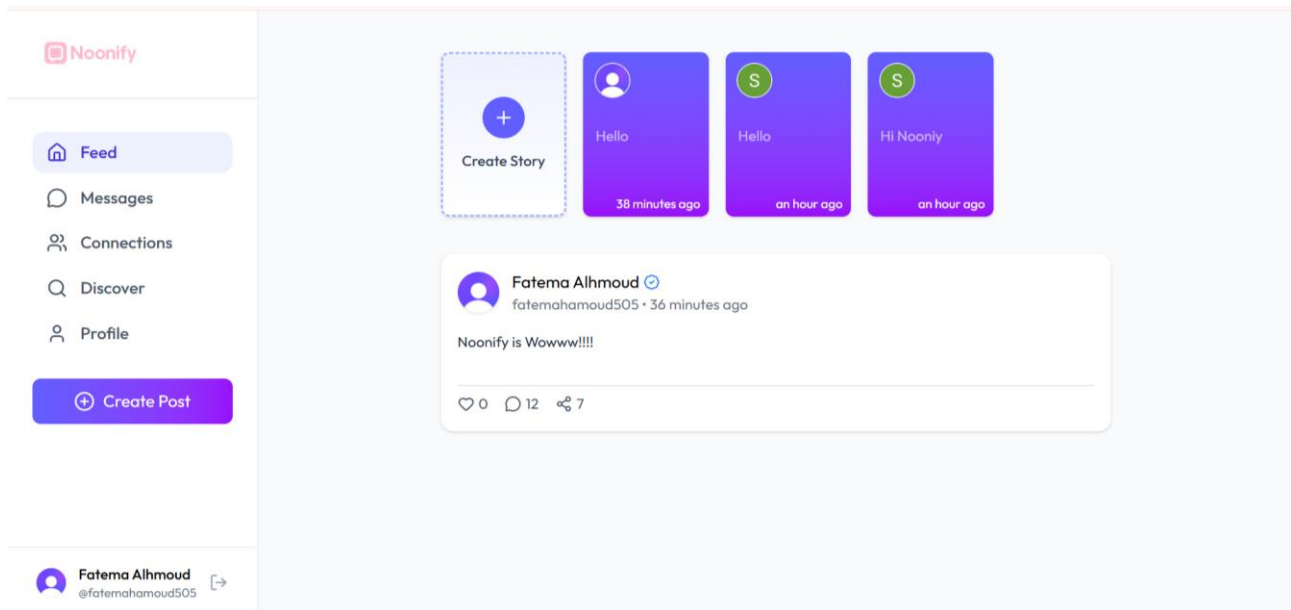
الشكل 4.14 : (1.6) إنشاء منشورات جديد



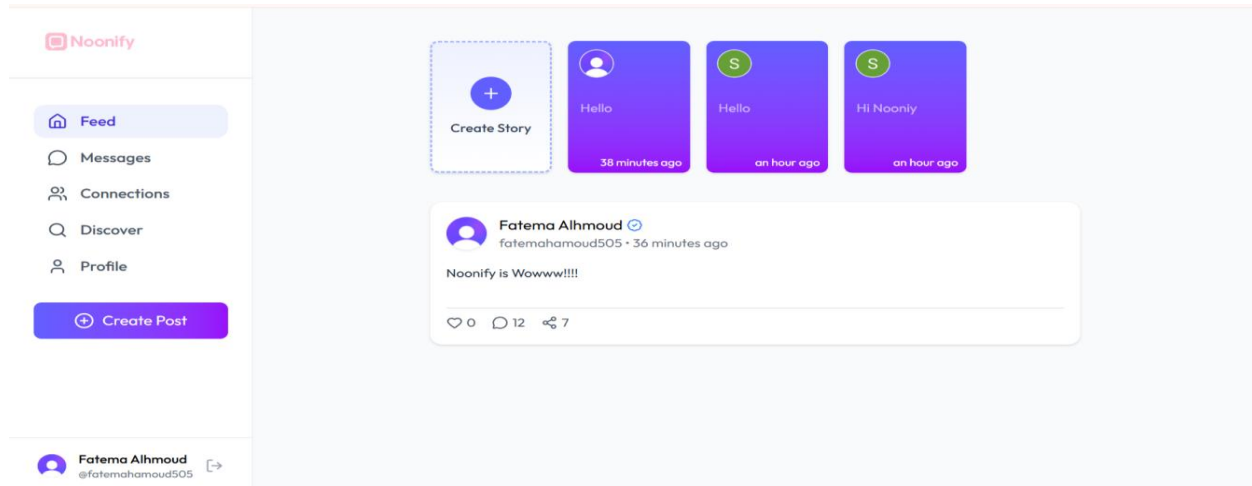
الشكل 4.15 : (1.7) عرض ملفات المستخدمين الآخرين



الشكل 4.16 : (1.8) تعديل الملف الشخصي



الشكل 4.17 : (1.9) عرض القصص



الشكل 4.18: (1.10) عرض منشورات الآخرين

4.2 الخاتمة

في ختام هذا المشروع، تم تصميم وتطوير منصة تواصل اجتماعي متكاملة تحت اسم **NOONIFY**، تهدف إلى محاكاة الخصائص الأساسية لشبكات التواصل الاجتماعي الحديثة مع مراعاة الجوانب الأكاديمية والعملية في آن واحد. اعتمد المشروع على معمارية **MERN Stack** التي تجمع بين الواجهة الأمامية التفاعلية باستخدام **React.js**، والواجهة الخلفية المبنية باستخدام **Node.js** و **Express.js**، إضافة إلى قاعدة البيانات **MongoDB**، مما ساهم في بناء نظام مرن، منظم، وقابل للتوسع.

نجح المشروع في تحقيق الأهداف المحددة له، حيث أتاح للمستخدمين إنشاء حساباتهم الشخصية، إدارة ملفاتهم الشخصية، مشاركة المحتوى النصي والمرئي، التفاعل مع المنشورات، استخدام نظام القصص، إضافة إلى التواصل عبر الرسائل الخاصة. كما تم تطبيق مفاهيم معمارية مهمة مثل **Client-Server Architecture** و **MVC Architecture**، الأمر الذي ساعد على تنظيم الكود، تحسين قابلية الصيانة، وتسهيل تطوير الميزات المستقبلية.

علاوةً على ذلك، ساهم هذا المشروع في تعزيز المهارات العملية في مجالات تطوير تطبيقات الويب الحديثة، العمل ضمن منهجية **Scrum**، وتحليل المتطلبات، مما جعله نموذجًا تطبيقيًا يجمع بين الجانبين النظري والعملية في هندسة البرمجيات ونظم المعلومات.

4.3 الآفاق المستقبلية (Future Work)

على الرغم من تحقيق المشروع لأهدافه الأساسية، إلا أن هناك العديد من التحسينات والتوسعات التي يمكن تنفيذها مستقبلاً لرفع كفاءة المنصة وتوسيع نطاق استخدامها، ومن أبرز هذه الآفاق:

1. تعزيز الأمان
إضافة تقنيات أمان متقدمة مثل المصادقة الثنائية (Two-Factor Authentication) وتحسين سياسات التشفير لحماية بيانات المستخدمين.
2. تحسين الأداء وقابلية التوسع
دعم تقنيات التخزين المؤقت (Caching) واستخدام خدمات سحابية متقدمة لتمكين المنصة من التعامل مع عدد أكبر من المستخدمين.
3. إضافة الذكاء الاصطناعي
دمج خوارزميات التوصية لاقتراح الأصدقاء والمحتوى المناسب بناءً على اهتمامات المستخدم.
4. تطوير تطبيقات الهواتف الذكية
إنشاء تطبيقات مخصصة لأنظمة **Android** و **iOS** لتوفير تجربة استخدام أكثر سلاسة.
5. توسيع خصائص التفاعل الاجتماعي
دعم البث المباشر (Live Streaming)، الإشعارات الفورية (Push Notifications)، وتحسين نظام التعليقات والتفاعلات.
6. تحسين تجربة المستخدم (UX/UI)
إجراء دراسات استخدام حقيقية لتحسين تصميم الواجهات وجعلها أكثر سهولة وجاذبية.

References

1.Books and Articles

1. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw–Hill Education.
2. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

2.Online Resources

4. MongoDB Inc. (n.d.). *MongoDB Documentation*. Retrieved from <https://www.mongodb.com/docs/>
5. React. (n.d.). *React Official Documentation*. Retrieved from <https://react.dev/>
6. Node.js Foundation. (n.d.). *Node.js Documentation*. Retrieved from <https://nodejs.org/en/docs>
7. Express.js. (n.d.). *Express Framework Documentation*. Retrieved from <https://expressjs.com/>

3.Tools and Software Documentation

8. Vite. (n.d.). *Vite – Next Generation Frontend Tooling*. Retrieved from <https://vitejs.dev/>
9. Mongoose. (n.d.). *Mongoose ODM Documentation*. Retrieved from <https://mongoosejs.com/docs/>
10. NodeMailer. (n.d.). *NodeMailer Documentation*. Retrieved from <https://nodemailer.com/about/>

4.Additional References

- 11.Scrum.org. (n.d.). *Scrum Guide*. Retrieved from <https://www.scrumguides.org/>
- 12.Mozilla Developer Network (MDN). (n.d.). *Web Technologies Documentation*. Retrieved from <https://developer.mozilla.org/>
- 13.ImageKit. (n.d.). *ImageKit Media Optimization Documentation*. Retrieved from <https://imagekit.io/docs>